

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»

Факультет інформатики та обчислювальної техніки

(повне найменування інституту, факультету)

Інформатики та програмної інженерії

(повна назва кафедри)

Курсова робота

з дисципліни Компоненти програмної інженерії

на тему

Веб-застосування організації навчальних
дисциплін

Виконав: студент III
курсу, групи

ІІІ-23 Подгорний Іван Русланович

(прізвище, ім'я, по батькові)

(підпис)

Керівник

ст. викладач, PhD, Головченко М.М.

посада, науковий ступінь, вчене звання, прізвище, і ім'я, по батькові

(підпис)

Члени комісії

ст. викладач, PhD, Головченко М.М.

посада, науковий ступінь, вчене звання, прізвище, і ім'я, по батькові

(підпис)

доцент Поперешняк С. В.

посада, науковий ступінь, вчене звання, прізвище, і ім'я, по батькові

(підпис)

доцент Зубик Л. В.

посада, науковий ступінь, вчене звання, прізвище, і ім'я, по батькові

(підпис)

Засвідчую, що у цій курсовій роботі немає
запозичень з праць інших авторів без
відповідних посилань.

Студент

(підпис)

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Рівень вищої освіти – перший (бакалаврський)

Спеціальність – 121 Інженерія програмного забезпечення

Освітньо-професійна програма – Інженерія програмного забезпечення
інформаційних систем

ЗАТВЕРДЖУЮ

керівник

(підпис) Максим ГОЛОВЧЕНКО
(ім'я прізвище)

“ ” _____ 2024 р.

ЗАВДАННЯ
на курсову роботу студенту

Подгорний Іван Русланович

(прізвище, ім'я, по батькові)

1. Тема роботи «Веб-застосування організації
навчальних дисциплін»

керівник роботи Головченко Максим Миколайович, PhD
(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

2. Термін подання студентом роботи « 22 » грудня 2024 року

3. Вихідні дані до роботи: технічне завдання

4. Зміст пояснювальної записки

1) Загальні положення: основні визначення та терміни, опис предметного середовища, огляд ринку програмних продуктів, постановка задачі.

2) Інформаційне забезпечення: вхідні дані, вихідні дані, опис структури бази даних.

3) Математичне забезпечення: змістовна та математична постановки задачі, обґрунтування та опис методу розв'язання.

4) Програмне та технічне забезпечення: засоби розробки, вимоги до технічного забезпечення, архітектура програмного забезпечення, побудова звітів.

5) Технологічний розділ: керівництво користувача, методика випробувань програмного продукту.

5. Перелік графічного матеріалу

1) Схема структурна варіантів використань

2) Схема структурна послідовностей

3) Схема бази даних

4) Схема структурна компонент

6. Консультанти розділів роботи

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв

7. Дата видачі завдання «12» вересня 2024 року _____

Календарний план

№ з/п	Назва етапів виконання курсової роботи	Термін виконання етапів роботи	Примітка
1	Вивчення рекомендованої літератури	20.10.2024	
2	Аналіз існуючих методів розв'язання задачі	04.11.2024	
3	Постановка та формалізація задачі	08.11.2024	
4	Аналіз вимог до програмного забезпечення	14.11.2024	
5	Алгоритмізація задачі	20.11.2024	
6	Моделювання програмного забезпечення	24.11.2024	
7	Обґрунтування використовуваних технічних засобів	30.11.2024	
8	Розробка архітектури програмного забезпечення	05.12.2024	
9	Розробка програмного забезпечення	10.12.2024	
10	Налагодження програми	11.12.2024	
11	Виконання графічних документів	12.12.2024	
12	Оформлення пояснювальної записки	14.12.2024	
13	Подання КР на основний захист	22.12.2024	

Студент

(підпис)

Іван ПОДГОРНИЙ

(ініціали, прізвище)

Керівник

(підпис)

Максим ГОЛОВЧЕНКО

(ініціали, прізвище)

АНОТАЦІЯ

Пояснювальна записка курсової роботи складається з п'яти розділів, містить 43 таблиць, 16 рисунків та 14 джерел – загалом 67 сторінки.

Курсова робота призначена для забезпечення ефективного ведення навчальних курсів та полегшення процесів створення та оцінювання завдань.

Метою розробки є підвищення зручності та ефективності ведення навчальних курсів

У розділі аналізу вимог були поставлені вимоги для програмного забезпечення.

У розділі моделювання програмного забезпечення було описано архітектуру програмного забезпечення та алгоритми вирішення прикладних задач.

У розділі аналіз якості були описані основні тест кейси, та стани системи після проведення тестування.

У розділі впровадження та супровід було описано процеси розгортання програмного забезпечення.

КЛЮЧОВІ СЛОВА: WEB APPLICATION, EDUCATION.

ABSTRACT

The explanatory note of the course work consists of five sections, contains 43 tables, 16 figures and 14 sources - a total of 67 pages.

The course work is designed to ensure effective conduct of training courses and facilitate the processes of creating and evaluating tasks.

The purpose of the development is to increase the convenience and efficiency of conducting training courses

In the requirements analysis section, requirements for the software were set.

In the software modeling section, the software architecture and algorithms for solving applied problems were described.

In the quality analysis section, the main test cases and system states after testing were described.

The implementation and maintenance section described the software deployment processes.

KEYWORDS: WEB APPLICATION, EDUCATION.

Відомість курсової роботи

[illegible]

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

керівник

_____ Максим ГОЛОВЧЕНКО

“ ____ ” _____ 2024 р.

ВЕБ-ЗАСТОСУВАННЯ ОРГАНІЗАЦІЯ НАВЧАЛЬНИХ КУРСІВ

Технічне завдання

КП.ІІ-23.045440.01.91

“ПОГОДЖЕНО”

Керівник роботи:

Максим ГОЛОВЧЕНКО

Виконавець:

Іван ПОДГОРНИЙ

Київ – 2024

ЗМІСТ

<u>1</u>	<u>НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ</u>	3
<u>2</u>	<u>ПІДСТАВА ДЛЯ РОЗРОБКИ</u>	4
<u>3</u>	<u>ПРИЗНАЧЕННЯ РОЗРОБКИ</u>	5
<u>4</u>	<u>ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ</u>	6
<u>4.1</u>	<u>Вимоги до функціональних характеристик</u>	6
<u>4.1.1</u>	<u>Користувацького інтерфейсу</u>	6
<u>4.1.2</u>	<u>Для користувача:</u>	9
<u>4.1.3</u>	<u>Для адміністратора системи (якщо він передбачений):</u>	9
<u>4.1.4</u>	<u>Додаткові вимоги:</u>	9
<u>4.2</u>	<u>Вимоги до надійності</u>	10
<u>4.3</u>	<u>Умови експлуатації</u>	10
<u>4.3.1</u>	<u>Вид обслуговування</u>	10
<u>4.3.2</u>	<u>Обслуговуючий персонал</u>	10
<u>4.4</u>	<u>Вимоги до складу і параметрів технічних засобів</u>	10
<u>4.5</u>	<u>Вимоги до інформаційної та програмної сумісності</u>	10
<u>4.5.1</u>	<u>Вимоги до вхідних даних</u>	10
<u>4.5.2</u>	<u>Вимоги до вихідних даних</u>	10
<u>4.5.3</u>	<u>Вимоги до мови розробки</u>	11
<u>4.5.4</u>	<u>Вимоги до середовища розробки</u>	11
<u>4.5.5</u>	<u>Вимоги до представленню вихідних кодів</u>	11
<u>4.6</u>	<u>Вимоги до маркування та пакування</u>	11
<u>4.7</u>	<u>Вимоги до транспортування та зберігання</u>	11
<u>4.8</u>	<u>Спеціальні вимоги</u> Error! Bookmark not defined.	
<u>5</u>	<u>ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ</u>	12
<u>5.1</u>	<u>Попередній склад програмної документації</u>	12
<u>5.2</u>	<u>Спеціальні вимоги до програмної документації</u>	12
<u>6</u>	<u>СТАДІЇ І ЕТАПИ РОЗРОБКИ</u>	13
<u>7</u>	<u>ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ</u>	14

НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ

Назва розробки: Веб-застосування організації навчальних курсів.

Галузь застосування:

Наведене технічне завдання поширюється на розробку програмного забезпечення для веб-застосування організації навчальних курсів, котра використовується для організації навчальних курсів на потреби навчальних закладів та призначена для спрощення проведення навчальних дисциплін. Додаток буде корисний начальним закладам, компаніям які надають освітні послуги та всім тим хто хоче реалізувати свій особистий навчальний простір.

ПІДСТАВА ДЛЯ РОЗРОБКИ

Підставою для розробки Веб-застосування організації навчальних курсів є індивідуальний навчальний план студента та РНП затверджена кафедрою інформатики та програмної інженерії Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського».

ПРИЗНАЧЕННЯ РОЗРОБКИ

Розробка призначена для забезпечення зручного та зрозумілого способу ведення навчальних курсів

Метою розробки є підвищення ефективності та якості навчального процесу для учнів, а також оптимізація та полегшення роботи для викладачів шляхом впровадження сучасних автоматизованих рішень.

ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Вимоги до функціональних характеристик

Програмне забезпечення повинно забезпечувати виконання наступних основних функцій:

Користувацького інтерфейсу

— Реєстрація (рис. 4.1)

StudyHub

Welcome back!

Full Name

Email

Password 

[Sign Up](#)

Or

 Continue with Google



Рисунок 4.1 – Сторінка реєстрації

— Логін. (рис. 4.2)

StudyHub

Welcome back!

Email

Password 

[Sign In](#)

Or

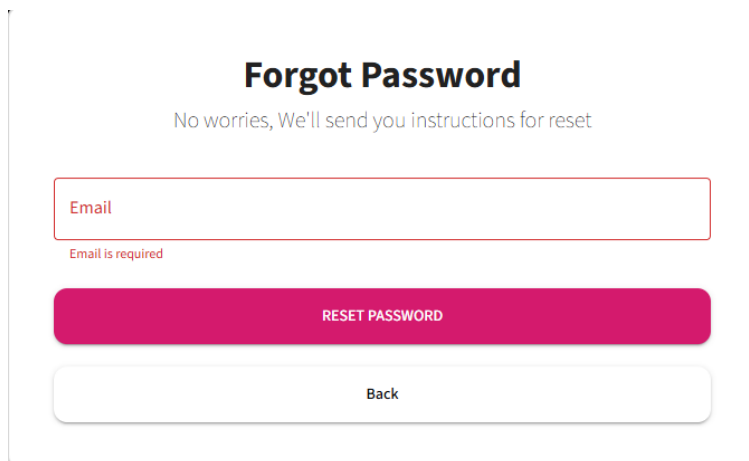
 Continue with Google

[Forgot password?](#)



Рисунок 4.2 – Сторінка входу

— Відновлення паролю через імейл (рис. 4.3)



Forgot Password

No worries, We'll send you instructions for reset

Email

Email is required

RESET PASSWORD

Back

Рисунок 4.3 – Сторінка для відновлення паролю

— Перегляд історії оцінок (рис. 4.4)

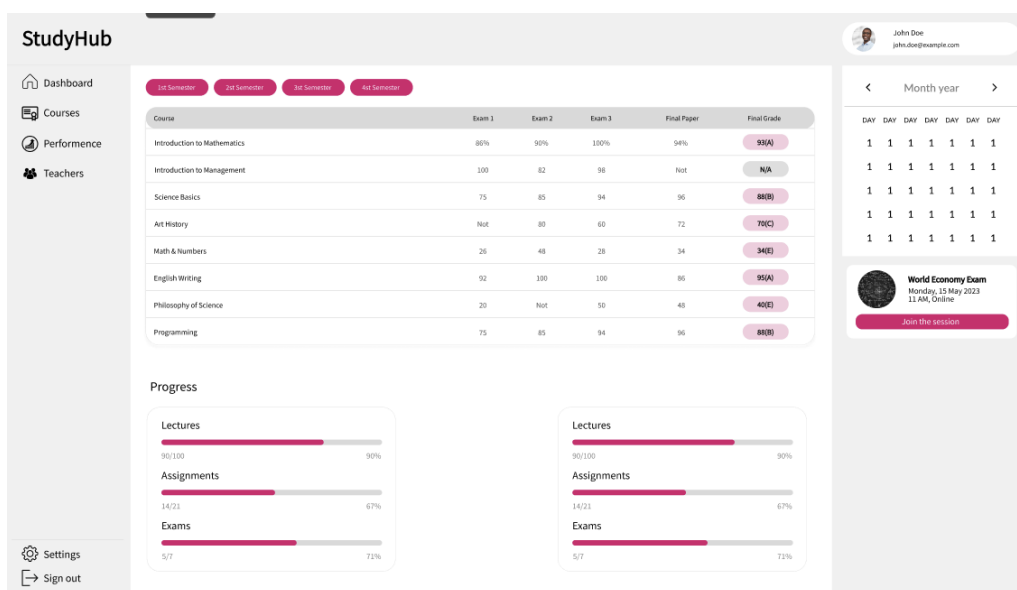


Рисунок 4.4 – Сторінка перегляду поточного прогресу

— Перегляд предметів, пошук предмету по назві (рис. 4.5)

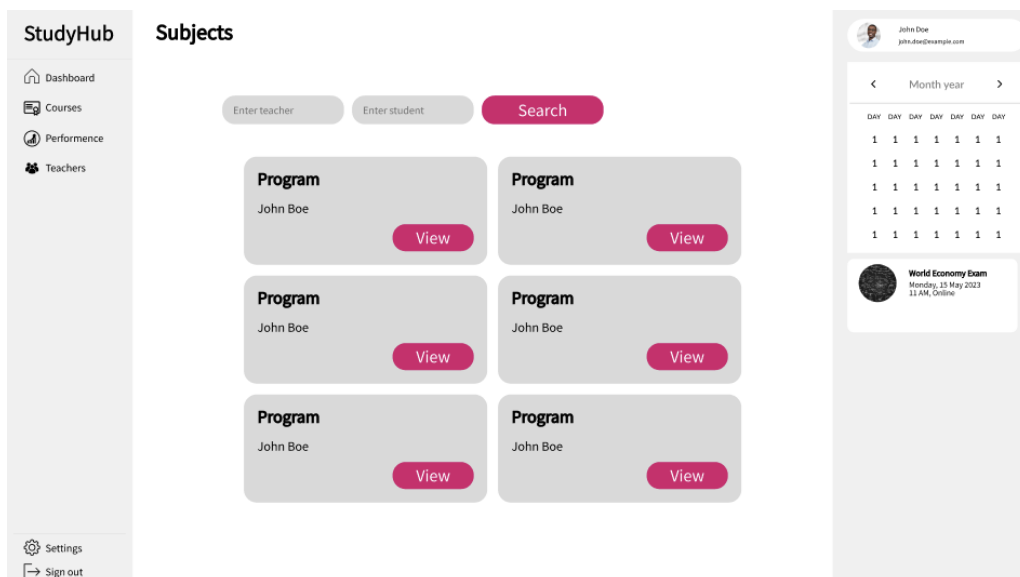


Рисунок 4.5 – Сторінка поточних предметів користувача

— Проходження та створення тестів (рис. 4.6)

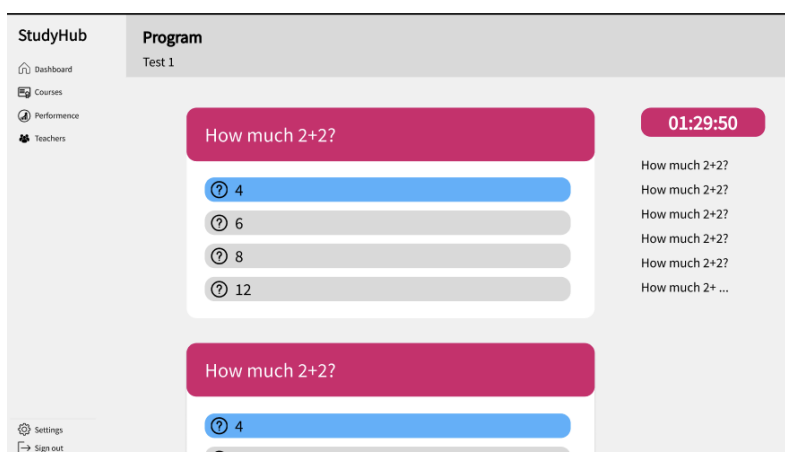


Рисунок 4.6 – Сторінка проходження тестів

- Перегляд поточного предмету з завданнями (рис. 4.7)

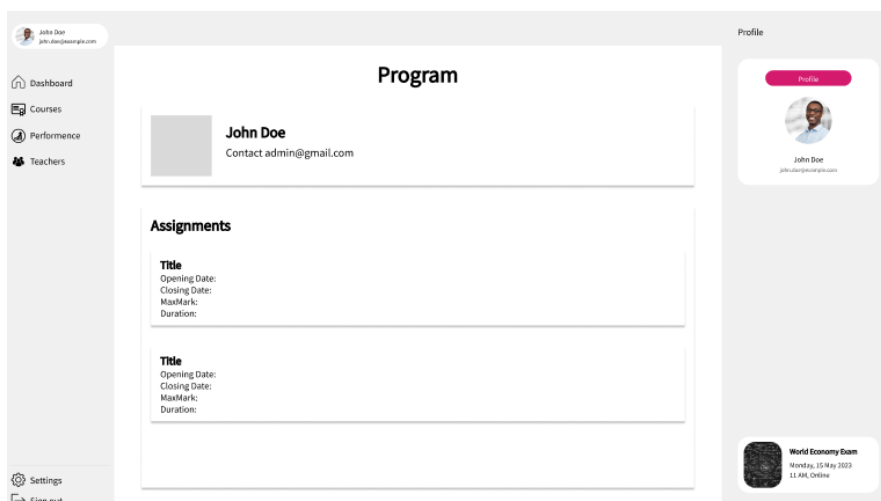


Рисунок 4.7 – Сторінка завдань на цьому предметі

Для користувача:

- Реєстрація через імейл та пароль
- Реєстрація через Google OAuth2
- Логін через імейл та пароль
- Логін через Google OAuth2
- Відновлення паролю через імейл
- Зміна фото профілю, імені користувача, імейлу
- Перегляд предметів
- Перегляд завдань

Роль Вчителя:

- Створення нових предметів та додавання здобувачів освіти на них
- Ведення та перегляд оцінок студентів
- Створення різних видів завдань з можливістю комбонування різних варіантів
- Можливість запросити інших викладачів та студента через імейл

Роль Студента:

- Проходження завдань
- Перегляд особистих оцінок

Для адміністратора системи (якщо він передбачений):

- Реєстрація через імейл та пароль
- Реєстрація через Google OAuth2
- Логін через імейл та пароль
- Логін через Google OAuth2
- Відновлення паролю через імейл
- Зміна фото профілю, імені користувача, імейлу
- Можливість видаляти предмети\завдання за певних причин
- Можливість запросити адміністратора через імейл

Вимоги до надійності

Передбачити контроль введення інформації та захист від некоректних дій користувача. Забезпечити цілісність інформації в базі даних.

Умови експлуатації

Умови експлуатації згідно СанПін 2.2.2.542 – 96.

Вид обслуговування

Вимоги до виду обслуговування не висуваються

Обслуговуючий персонал

Вимоги до обслуговуючого персоналу не висуваються

Вимоги до складу і параметрів технічних засобів

Програмне забезпечення повинно функціонувати на IBM-сумісних персональних комп'ютерах.

Мінімальна конфігурація технічних засобів:

- тип процесору: Intel Core i3;
- об'єм ОЗП: 4 Гб;
- підключення до мережі Інтернет зі швидкістю від 10 мегабіт;

Рекомендована конфігурація технічних засобів:

- тип процесору: Intel Core i5;
- об'єм ОЗП: 6 Гб;
- підключення до мережі Інтернет зі швидкістю від 50 мегабіт;

Вимоги до інформаційної та програмної сумісності

Програмне забезпечення повинно працювати під управлінням операційних систем сімейства WIN32 або Unix. На комп'ютері має бути встановлений .NET 7 та Node.js 18.

Вимоги до вхідних даних

Вхідні дані повинні бути представлені в наступному форматі: HTTP запити на ендпоінти API у форматі JSON

Вимоги до вихідних даних

Результати повинні бути представлені в наступному форматі: відповіді у форматі JSON

Вимоги до мови розробки

Розробку виконати на мові програмування C# та React

Вимоги до середовища розробки

Розробку виконати на платформі .NET 7 та React.ts

Вимоги до маркування та пакування

Вимоги до маркування та пакування не висуваються.

Вимоги до транспортування та зберігання

Вимоги до транспортування та зберігання не висуваються.

ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ

Попередній склад програмної документації

У склад супроводжувальної документації повинні входити наступні документи на аркушах формату А4:

- технічне завдання;
- пояснювальна записка;
- текст програми.

Спеціальні вимоги до програмної документації

Програмні модулі, котрі розробляються, повинні бути задокументовані, тобто тексти програм повинні містити всі необхідні коментарі.

СТАДІЇ І ЕТАПИ РОЗРОБКИ

№	Назва етапу	Строк	Звітність
1.	Вивчення літератури за тематикою роботи	12.09.2024	
2.	Розробка технічного завдання	15.09.2024	Технічне завдання
3.	Аналіз вимог та уточнення специфікацій	21.09.2024	Специфікації програмного забезпечення
4.	Проектування структури програмного забезпечення, проектування компонентів	24.09.2024	Схема структурна програмного забезпечення та специфікація компонентів
5.	Програмна реалізація програмного забезпечення	11.10.2024	Тексти програмного забезпечення
6.	Тестування програмного забезпечення	04.11.2024	Тести, результати тестування
7.	Розробка матеріалів текстової частини роботи	09.10.2024	Пояснювальна записка
8.	Розробка матеріалів графічної частини роботи	02.12.2024	Графічний матеріал проєкту
9.	Оформлення технічної документації роботи	12.12.2024	Технічна документація

ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ

Тестування розробленого програмного продукту виконується відповідно до “Розділу тестування програмного забезпечення”.

Пояснювальна записка до курсової роботи

на тему: Веб-застосування організації навчальних курсів.

КП.ІІ-23.045440.02.81

Київ – 2024

ЗМІСТ

<u>ВСТУП</u>	6
<u>1 ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ</u>	7
<u>1.1 Аналіз предметної області</u>	7
<u>1.2 Аналіз існуючих рішень</u>	7
<u>1.2.1 Аналіз відомих програмних продуктів</u>	7
<u>1.2.2 Аналіз відомих алгоритмічних та технічних рішень</u>	9
<u>1.3 Опис бізнес-процесів</u>	9
<u>1.4 Постановка задачі</u>	13
Висновки до розділу	13
<u>2 РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ</u>	16
<u>2.1 Варіанти використання програмного забезпечення</u>	16
<u>2.2 Аналіз системних вимог</u>	23
<u>2.3 Розроблення функціональних вимог</u>	24
<u>2.4 Розроблення нефункціональних вимог</u>	29
Висновки до розділу	29
<u>3 КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ</u>	32
<u>3.1 Архітектура програмного забезпечення</u>	32
<u>3.2 Обґрунтування вибору засобів розробки</u>	38
<u>3.3 Конструювання програмного забезпечення</u>	39
<u>3.4 Аналіз безпеки даних</u>	49
Висновки до розділу	49
<u>4 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ</u>	52
<u>4.1 Аналіз якості ПЗ</u>	52
<u>4.2 Опис процесів тестування</u>	52
<u>4.3 Опис контрольного прикладу</u>	57
Висновки до розділу	60
<u>5 РОЗГОРТАННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ</u>	62

5.1	Розгортання програмного забезпечення	62
5.2	Супровід програмного забезпечення Error! Bookmark not defined.	
5.3	Повна інструкція користувача	62
	Висновки до розділу	63
	ВИСНОВКИ	65
	СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	67
	ДОДАТКИ	68

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

IDE	– Integrated Development Environment – інтегроване середовище розробки.
API	– Application programming interface, прикладний програмний Інтерфейс
SDK	– Software development kit
IT	– Інформаційні технології
ER	– Entity-Relation diagram
OC	– Операційна система.
БД	– База даних.

ВСТУП

У сучасному світі з появою глобалізації зростає кількість інформації та нових технологій, які потребують опанування. Технологічний прогрес та поширення Інтернету відкрили нові значно ширші можливості ведення навчального процесу в школах, закладах вищої освіти, тощо.

Окрім традиційних методів викладання, використання веб-застосунків дозволяє створювати інтерактивне середовище, яке сприяє залученню студентів, гнучкості навчання та персоналізації освітнього контенту. Такі інструменти, як платформи для організації курсів, надають можливість ефективно управляти навчальними матеріалами, відстежувати прогрес учасників, а також забезпечувати зворотний зв'язок у реальному часі.

Важливо зазначити, що розробка веб-застосунків для навчання сприяє подоланню географічних бар'єрів, забезпечуючи доступ до якісної освіти незалежно від місця проживання. До того ж, такі системи можуть інтегрувати сучасні технології.

Мета даної курсової роботи — дослідити особливості розробки веб-застосунків для організації навчальних курсів, оцінити їхній вплив на ефективність освітнього процесу та запропонувати оптимальні підходи до їхнього впровадження.

ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ

Аналіз предметної області

Сьогодні, не можна уявити здобуття освіти без використання веб-застосунків, які дозволяють автоматизувати, та децентралізувати здобуття освіти, спрощувати комунікацію між студентами та викладачами, а також забезпечувати доступність до навчальних матеріалів. Ці системи використовуються в багатьох освітніх структурах, надаючи можливість проводити навчання у гнучкому форматі, що відповідає потребам студентів та викладачів у різних обставинах.

Сучасні веб-застосунки, такі як Moodle, Google Classroom, Naurok та інші, базуються на інтеграції функціональності для управління курсами, моніторингу успішності, створення завдань та оцінювання. Однак такі рішення інколи мають недоліки, зокрема у гнучкості та зрозумілості налаштувань або масштабуванні для великих навчальних закладів із тисячами користувачів.

У рамках курсової роботи передбачається розробка веб-застосунку, що забезпечить просте управління курсами, інтерактивне навчання та розширені функції для співпраці між студентами і викладачами. Особливий акцент буде зроблено на створенні user friendly інтерфейсу для покращення користувацького досвіду.

Аналіз існуючих рішень

Проаналізуємо відоме на сьогодні алгоритмічне забезпечення у даній області та технічні рішення, що допоможуть у реалізації Веб-застосування організації навчальних курсів - StudyHub. Далі будуть розглянуті готові програмні рішення, допоміжні програмні засоби та засоби розробки.

Аналіз відомих програмних продуктів

Для порівняння розробки з аналогом можна скористатись таблицею 1.1.

Таблиця 1.1 – Порівняння з аналогом

Функціонал	StudyHub	Google Classroom	Naurok	Moodle
Реєстрація та авторизація через Google Auth 2.0	+	+	+	-
Відновлення паролю через імейл	+	+	+	+
Створення та ведення предмету\курсу	+	+	-	+
Можливість створення різних варіантів для питань на тест	+	-	-	+
Зручний перегляд оцінок як для вчителя, так і для учнів	+	+/-	-	+
Додавання користувачів на платформу шляхом запрошення їх через імейл	+	-	-	+/-

У межах аналізу існуючих програмних продуктів для порівняння було обрано три платформи: Google Classroom, Naurok та Moodle. Результати дослідження функціональних можливостей вказали, що лише Moodle частково відповідає вимогам функціоналу, закладеного в розробці курсової роботи, тоді як Classroom та Naurok демонструють суттєві обмеження. Зокрема, Moodle не підтримує реєстрацію через Google OAuth 2.0, що знижує зручність для користувачів. Платформа Naurok не забезпечує функції створення та адміністрування навчальних предметів, а функціонал створення варіантів завдань реалізований лише частково в Moodle.

Щодо можливості перегляду оцінок, вона якісно реалізована в Moodle, тоді як у Classroom та Naurok вона доступна лише для вчителів. Базові функції, такі як авторизація, реєстрація та відновлення паролю, присутні на всіх платформах. Водночас StudyHub вирізняється своєю закритістю, що передбачає можливість запрошення користувачів для подальшої реєстрації, забезпечуючи додатковий контроль доступу до платформи.

1.2.2 Аналіз відомих алгоритмічних та технічних рішень

Створення тестів є головним елементом системи, що забезпечує створення адаптивних завдань для користувачів. У системі підтримуються наступні сутності:

- Assignment – основний тест.
- AssignmentTask – конкретне завдання в тесті.
- AssignmentVariant – варіант завдання, який включає набір питань.
- AssignmentOption – опції тесту.

Механізм роботи білдера тестів передбачає динамічне створення завдань і варіантів. Головною особливістю є можливість створення банку питань для кожного завдання, з якого система рандомним чином вибирає питання для користувача.

Принципи роботи:

- Завдання можуть мати лише варіанти одного типу (наприклад, Select або Multi).
- Для простих тестів можливий вибір лише одного варіанта завдання.
- Питання вибираються випадковим чином із банку питань, забезпечуючи унікальність кожного тесту.

Реалізація таймера та кешування:

- При старті тесту сервер генерує унікальний ідентифікатор та запускає таймер.
- Клієнт отримує завдання та зберігає його у кеші. У разі перезавантаження сторінки клієнт перевіряє хеш або відправляє запит для відновлення тесту.
- У разі відсутності даних у кеші чи хеші сервер повертає помилку 400, запобігаючи несанкціонованим спробам повторного проходження тесту.

Такі підходи забезпечують динамічність, зручність та захищеність процесу створення та проходження тестів.

Система запрошень є важливою складовою безпеки та зручності платформи, дозволяючи адміністратору чи вчителю запрошувати користувачів до курсу.

Принцип роботи:

1. Адміністратор вказує електронну пошту користувача, якого потрібно запросити.
2. Запрошення зберігається в окремій таблиці, яка містить:
 - електронну пошту;
 - унікальний токен;
 - час створення запрошення.
3. Користувач отримує лист із запрошенням на пошту та переходить за посиланням.

4. При реєстрації система звіряє токен із даними в таблиці. Якщо токен збігається, запрошення підтверджується, а запис видаляється.

Додаткові функції:

- Якщо запрошення не було прийнято протягом 7 днів, бекграунд-джоб автоматично видаляє запис. Це запобігає спробам доступу до платформи через застарілі токени.
- У разі видалення запрошення користувачу потрібно надіслати нове запрошення.

Ця система забезпечує високий рівень контролю над доступом до платформи, виключаючи можливість реєстрації без підтвердження адміністратора або іншого вчителя.

Опис бізнес-процесів

Для опису бізнес процесу використовується BPMN модель (рисунок 1.1).

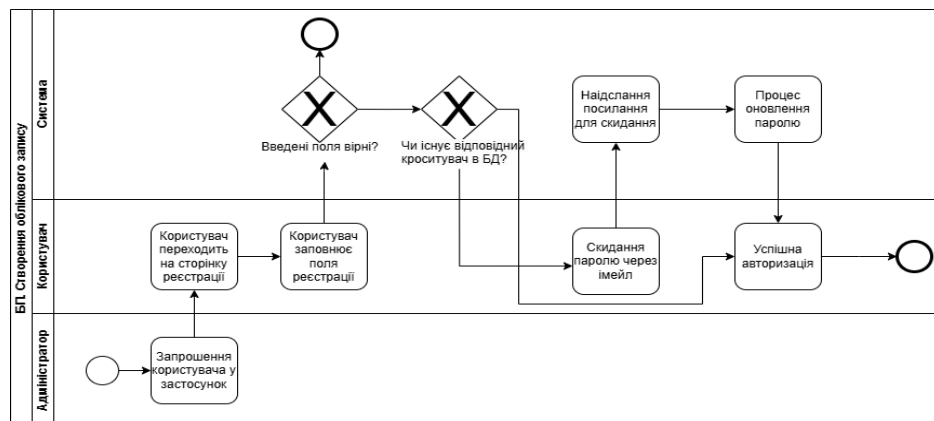


Рисунок 1.1 – Схема створення облікового запису користувача

Опис послідовності створення облікового запису користувача(рис. 1.1):

- Адміністратор запрошує користувача у застосунок
- Користувач проходить за посилання яке прийшло на пошту
- Користувач заповнює поля реєстрації;
- Якщо введені поля, не відповідають шаблону заповнення на

- клієнтській стороні, відповідні поля підсвічуються помилкою;
- Якщо введені поля правильні та існують в БД користувачів, перехід до сторінки авторизації, інакше перехід до сторінки створення облікового запису;
- Користувач заповнює поля реєстрації;
- Після успішного заповнення всіх полів користувач натискає кнопку "Зареєструватися";
- Користувач може увійти до системи, використовуючи електронну пошту та пароль після підтвердження адреси;
- Користувач може додати додаткову інформацію до профілю;

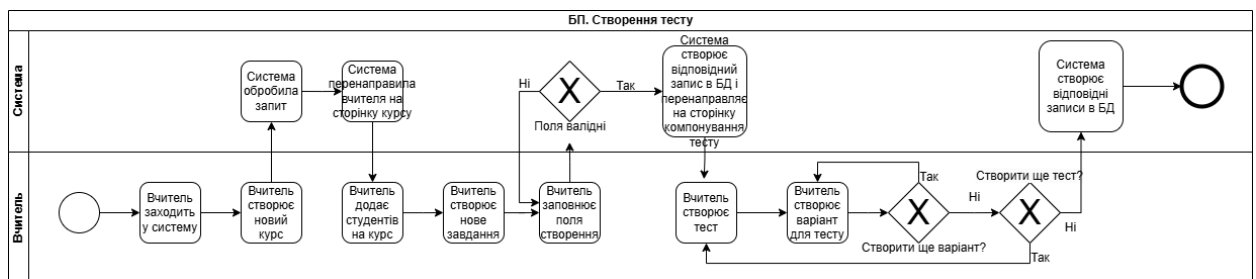


Рисунок 1.2 – Схема створення завдання для студента

- Вчитель заходить у застосунок
- Вчитель створює новий предмет
- Система оброблює запит та перенаправляє вчителя на сторінку предмета
- Вчитель додає студентів на предмет
- Вчитель створює нове завдання
- Вчитель зповнює поля для створення завдання
- Система валідує введені поля, і при невдалій валідації висвічує помилку, а при вдалий перенаправляє вчителя на сторінку компонування тесту
- Вчитель створює тести
- Вчитель створює певну кількість варіант для кожного тесту

– Система оброблює запит та записує у БД;

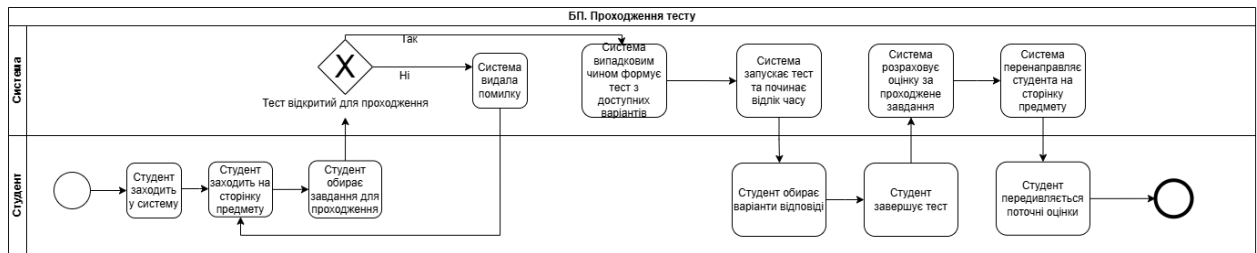


Рисунок 1.3 – Схема проходження завдання студентом

- Студент заходить у застосунок
- Студент заходить на сторінку предмету
- Студент обирає завдання
- Система, якщо завдання відкрите для проходження, komponує випадковим чином тест з наявних варіантів завдань
- Система починає запускате тест та починає зворотній відлік
- Студент обирає варіанти відповіді
- Студент завершує тест
- Система розраховує поточну оцінку студента за тест та записує її
- Система перенаправлює студента на сторінку предмету
- Студент передивляється поточні оцінки;

Постановка задачі

У межах цієї роботи буде створено платформу для управління освітніми дисциплінами, яка забезпечить зручність та функціональність для вчителів, студентів та адміністрації навчальних закладів. Основною метою є розробка системи, яка включатиме можливість реєстрації та авторизації користувачів через класичну модель з використанням імейлу та паролю, а також підтримку Google OAuth2. Для відновлення доступу передбачено механізм скидання паролю через імейл. Користувачі матимуть можливість змінювати свої персональні дані, зокрема фото профілю, ім'я та адресу електронної пошти.

Система забезпечуватиме функціональність для різних ролей. Вчителі зможуть створювати нові предмети, додавати студентів до них, створювати та керувати завданнями, а також переглядати й вести оцінки студентів. Особливої уваги заслуговує механізм створення тестів, який дозволить компонувати завдання з різними варіантами відповідей. Кожне завдання може включати банк питань, із якого випадковим чином обиратимуться питання для кожного користувача. Ця функція дозволить забезпечити унікальність тестів та їх адаптивність. Крім того, вчителі зможуть запрошувати інших викладачів до роботи на платформі через електронну пошту.

Студенти, у свою чергу, отримають змогу проходити завдання, переглядати свої оцінки та отримувати автоматизований зворотний зв'язок. Система передбачає використання таймерів для обмеження часу на виконання завдань, а також механізм кешування, що забезпечить відновлення прогресу у разі перезавантаження сторінки. У разі спроби повторного доступу до тесту система відповідно оброблятиме запити, виключаючи можливість маніпуляцій із тестуванням.

Для адміністратора платформа включатиме можливість створення й управління контентом, видалення предметів або завдань за потреби, а також надсилання запрошень адміністраторам і вчителям через імейл. Важливим аспектом є механізм управління запрошеннями: токени для реєстрації зберігатимуться в базі даних і автоматично видалятимуться через тиждень, якщо запрошення не буде прийняте. Це забезпечить високий рівень безпеки та контроль доступу до платформи.

Запропоноване рішення має на меті створення інструменту, який спростить управління освітнім процесом, забезпечить гнучкість і зручність для користувачів, а також дозволить ефективно впроваджувати інноваційні методи навчання у навчальних закладах.

Висновки до розділу

У цьому розділі курсової роботи було здійснено аналіз предметної області та наявних рішень для розробки платформи управління освітніми дисциплінами. Розглянуто основні функціональні можливості таких систем, включаючи адміністрування предметів, управління завданнями, облік оцінок, а також можливість інтеграції з зовнішніми сервісами, такими як Google OAuth2 для реєстрації та авторизації. Особливу увагу приділено автоматизації освітніх процесів і зручності взаємодії між викладачами, студентами та адміністраторами.

Було проведено детальний аналіз алгоритмів і підходів до реалізації ключових функцій, серед яких створення тестів із рандомізацією питань, можливість відстеження прогресу студентів та індивідуалізація навчального процесу через персоналізовані завдання. У рамках дослідження також виокремлено найбільш ефективні підходи до побудови бізнес-логіки системи та взаємодії між користувачами.

Завершенням розділу стало моделювання бізнес-процесів для ключових функцій системи, таких як реєстрація користувачів, створення предметів і тестів, а також проходження завдань студентами. Ці процеси представлені у вигляді UML-діаграм, які відображають взаємодію викладачів, студентів та системи. Діаграми забезпечують структурований підхід до розуміння роботи платформи та її компонентів.

Таким чином, розділ формує теоретичну базу для подальшої розробки системи, описує основні етапи реалізації функціоналу та визначає задачі, необхідні для досягнення цілей курсової роботи.

РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Варіанти використання програмного забезпечення

Головною функцією програмного забезпечення є створення завдання для студентів, створення варіантів тесту, більше функцій можна побачити на рисунку 2.1.

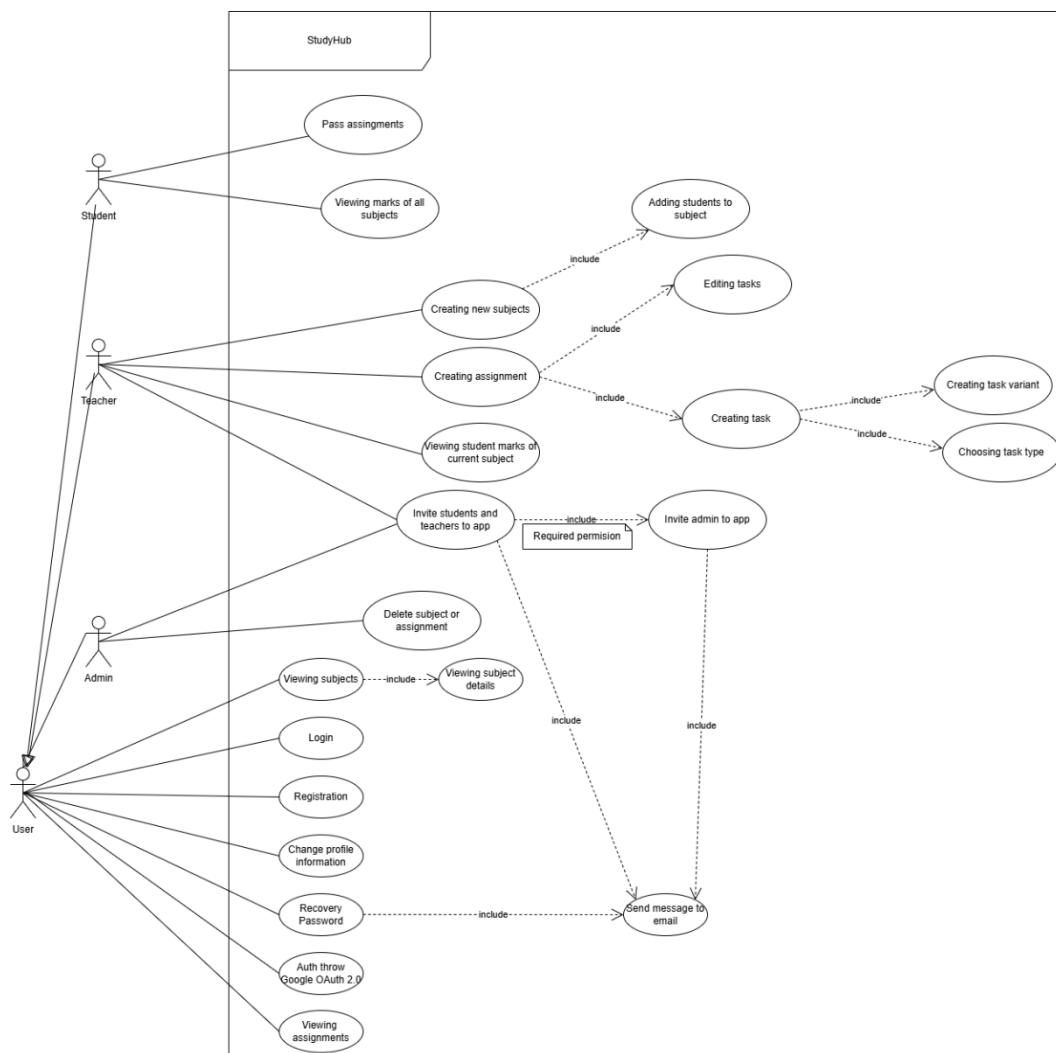


Рисунок 2.1 – Діаграма варіантів використання

В таблицях 2.2 - 2.14 наведені варіанти використання програмного забезпечення.

Таблиця 2.2 - Варіант використання UC-1

Use case name	Реєстрація користувача
Use case ID	UC-01

Goals	Реєстрація нового користувача в системі
Actors	Гість (неzareєстрований користувач)
Trigger	Користувач бажає зареєструватися
Pre-conditions	Користувачу на пошту прийшло запрошення на реєстрацію
Flow of Events	Користувач переходить на сторінку реєстрації. В поля для реєстрації вводяться відповідні дані: пошта користувача, пароль в системі, та його повтор для підтвердження або реєструється через Google OAuth 2.0. Після заповнення даних користувач натискає кнопку реєстрації. Після цього з'являється повідомлення про успішну реєстрацію, і користувач перенаправляється на сторінку входу.
Extension	Якщо якесь конкретне поле введено некоректно, то воно підсвічується червоним надписом.
Post-Condition	Створення сторінки користувача, перехід на сторінку входу

Таблиця 2.3 - Варіант використання UC-2

Use case name	Логін
Use case ID	UC-02
Goals	Логін користувача в системі
Actors	Гість
Trigger	Користувач бажає увійти в аккаунт
Pre-conditions	Користувач має обліковий запис
Flow of Events	Користувач переходить на сторінку логіну. В поля для реєстрації вводяться відповідні дані: пошта користувача, пароль в системі або користувач логінується через Google OAuth 2.0. Після заповнення даних користувач натискає кнопку увійти. Після цього з'являється повідомлення про успішне входження в аккаунт, і користувач перенаправляється на сторінку входу.
Extension	Якщо якесь конкретне поле введено некоректно, то воно підсвічується червоним надписом.

Post-Condition	Користувач увійшов в аккаунт, перехід на сторінку входу
----------------	---

Таблиця 2.4 - Варіант використання UC-3

Use case name	Логін
Use case ID	UC-03
Goals	Логін користувача в системі
Actors	Гість
Trigger	Користувач бажає увійти в аккаунт
Pre-conditions	Користувач має обліковий запис
Flow of Events	Користувач переходить на сторінку логіну. В поля для реєстрації вводяться відповідні дані: пошта користувача, пароль в системі або користувач логіниться через Google OAuth 2.0. Після заповнення даних користувача натискає кнопку увійти. Після цього з'являється повідомлення про успішне входження в аккаунт, і користувач перенаправляється на сторінку входу.
Extension	Якщо якесь конкретне поле введено некоректно, то воно підсвічується червоним надписом.
Post-Condition	Користувач увійшов в аккаунт, перехід на сторінку входу

Таблиця 2.5 - Варіант використання UC-4

Use case name	Перегляд поточних предметів
Use case ID	UC-04
Goals	Перегляд предметів користувача
Actors	User (будь-який користувач)
Trigger	Користувач бажає переглянути предмети
Pre-conditions	Користувач увійшов в систему
Flow of Events	Користувач переходить на сторінку предметів, де відображаються список усіх поточних предметів. Для

	викладача та адміна це предмети які вони ведуть, для студентів – їхні дисципліни для проходження
Extension	У разі відсутності предмету для студента буде виведено повідомлення „У немає поточних дисциплін,, , а для викладача та адміна буде можливість створити дисципліну
Post-Condition	Користувач бачить список предметів

Таблиця 2.6 - Варіант використання UC-5

Use case name	Зміна деталей профілю
Use case ID	UC-05
Goals	Зміна деталей профілю користувача
Actors	User (будь-який користувач)
Trigger	Користувач бажає змінити особисті данні
Pre-conditions	Користувач увійшов в систему
Flow of Events	Користувач переходить на сторінку профілю, де відображаються особисті данні. Користувач міняє бажані поля, при бажанні може змінити аватарку і натискає кнопку „Зберегти,,
Extension	У разі неправильного формату аватара або невалідних полей висвітиться помилка
Post-Condition	Користувач змінює особисті данні, та на екран виводиться повідомлення „Дані були оновлені успішно,,

Таблиця 2.7 - Варіант використання UC-6

Use case name	Запрошення користувачів у систему
Use case ID	UC-06
Goals	Запрошення нових користувачів у систему
Actors	Teacher, Admin
Trigger	Користувач бажає запросити нових користувачів

Pre-conditions	Користувач увійшов в систему під привілейованою роллю
Flow of Events	Користувач переходить на сторінку запрошень, де є можливість запросити інших користувачів у систему. Користувач у відповіднє поле вводить імейл адреса усих кого він хоче запросити, вибирає їм ролі під якими вони будуть запрошені (Для ролі вчителя вимкнута можливість запросити адміна), і натискає кнопку „Запросити,,
Extension	У разі виникнення помилки користувач буде повідомлен відповідним меседжом
Post-Condition	Користувач отримує повідомлення „Запрошення було розіслане успішно,, , а запрошеним користувачам прийшли запрошення у систему відповідно

Таблиця 2.8 - Варіант використання UC-7

Use case name	Видалення предмету
Use case ID	UC-07
Goals	Видалення предмету зі списку
Actors	Admin, Teacher
Trigger	Користувач бажає видалити предмет
Pre-conditions	Користувач увійшов в систему під привілейованою роллю, створення відповідна дисципліна для видалення
Flow of Events	Користувач переходить на сторінку предметів, обирає той предмет який бажає видалити, і натискає на відповідну іконку
Extension	Якщо з якихось причин це не вдалося зробити – користувачу видасть відповідну помилку
Post-Condition	Користувач успішно видаляє предмет. Предмет пропадає зі списку та виводиться на екран відповідне повідомлення що видалення пройшло успішно.

Таблиця 2.9 - Варіант використання UC-8

Use case name	Додавання студентів на дисципліну
---------------	-----------------------------------

Use case ID	UC-08
Goals	Додати студентів на дисципліну
Actors	Teacher
Trigger	Вчитель хоче додати студентів на
Pre-conditions	Користувач увійшов в систему під відповідною роллю, в системі наявні користувачі з роллю „Студент,,
Flow of Events	Користувач переходить на сторінку дисципліни, та натискає кнопку „Додати студентів,,. Після цього вчитель вводить відповідні імейли студентів яких він хоче додати, та натискає кнопку „Додати,,
Extension	У разі неправильного імейлу студента - вчитель буде повідомлен про помилку в написанні
Post-Condition	Студенти успішно додані на курс. Вчитель бачить оновлений список студентів на курсі разом з доданими

Таблиця 2.10 - Варіант використання UC-9

Use case name	Створення завдання
Use case ID	UC-09
Goals	Створити завдання для проходження студентам
Actors	Teacher
Trigger	Вчитель хоче оцінити знання студентів з дисципліни шляхом впровадження завдання з відповідної теми
Pre-conditions	Користувач увійшов в систему під відповідною роллю та створення відповідна дисципліна
Flow of Events	Вчитель переходить на сторінку дисципліни, та натискає кнопку „Додати завдання,,. Після цього вчитель вводить у відповідні поля деталі завдання: назву, кількість балів, дедлайни та час виконання. Вчитель переходить на сторінку завдання та створює тести. Вчитель обирає тип тесту та заповнює відповідні поля з питанням та варіантами відповідей. Вчитель на одне завдання може створити декілька варіантів. Після завершення компонування завдання, вчитель натискає кнопку зберегти.
Extension	У разі невалідних даних у створенні деталей до завдання, таких як: неправильно виставлений дедлайн, час

	проходження завдання більший за час відкриття та закриття завдання – Вчителю буде виводитися помилка та підсвічуватися відповідні поля для виправлення. При збереженні завдання, якщо виникне помилка, вчителю побачить відповідне повідомлення
Post-Condition	Вчитель успішно створив завдання. Відповідне завдання створене та може бути побачине студентами данного курсу. Тести для цього завдання успішно збережені та можуть бути пройдені відповідними студентами данного курсу.

Таблиця 2.11 - Варіант використання UC-10

Use case name	Перегляд оцінок студентів
Use case ID	UC-10
Goals	Переглянути оцінки студентів
Actors	Teacher
Trigger	Вчитель хоче побачити поточні результати студентів на конкретній дисципліні
Pre-conditions	Користувач увійшов в систему під відповідною роллю. На дисципліні наявні студенти.
Flow of Events	Вчитель переходить на сторінку дисципліни, та натискає на кнопку „Передивитися оцінки,.. Користувач переходить на сторінку з оцінками студентів цього курсу
Extension	У разі виникнення помилки вчителю прийде відповідне повідомлення
Post-Condition	Вчитель успішно продивився усі оцінки студентів.

Таблиця 2.12 - Варіант використання UC-11

Use case name	Проходження тесту
Use case ID	UC-11
Goals	Проходження тесту
Actors	Student

Trigger	Студент хоче пройти поточне завдання
Pre-conditions	Студент увійшов у систему. Вчитель створив завдання у відповідній дисципліні.
Flow of Events	Студент переходить на сторінку дисципліни, та натискає на відповідне завдання. Студент переходить на відповідну сторінку завдання та проходить тест. По закінченню натискає кнопку „Завершити тест,,
Extension	У разі, якщо завдання ще не відкрито, студенту буде видано відповідну помилку. При певних проблемах зі збереженням тесту студенту буде показано відповідну помилку
Post-Condition	Студент успішно пройшов тест, та одразу порахувало результат проходження.

Таблиця 2.13 - Варіант використання UC-12

Use case name	Перегляд оцінок студентом
Use case ID	UC-12
Goals	Переглянути поточні оцінки студента
Actors	Student
Trigger	Студент хоче побачити поточний результат за усі дисципліни
Pre-conditions	Користувач увійшов в систему під відповідною роллю. Студент наявний на деяких дисциплінах. Вчитель створив завдання
Flow of Events	Студент переходить на сторінку оцінок
Extension	У разі виникнення помилки студенту прийде відповідне повідомлення
Post-Condition	Студент успішно передивився свої оцінки.

Аналіз системних вимог

Програмне забезпечення повинно функціонувати на IBM-сумісних персональних комп'ютерах.

Мінімальна конфігурація технічних засобів:

- тип процесору: Intel Core i3;

- об'єм ОЗП: 4 Гб;
- підключення до мережі Інтернет зі швидкістю від 10 мегабіт;

Рекомендована конфігурація технічних засобів:

- тип процесору: Intel Core i5;
- об'єм ОЗП: 6 Гб;
- підключення до мережі Інтернет зі швидкістю від 50 мегабіт;

Програмне забезпечення повинно працювати під управлінням операційних систем сімейства WIN32 або Unix. На комп'ютері має бути встановлений .NET 7 та Node.js 18.

Розроблення функціональних вимог

Програмне забезпечення розділене на модулі. Кожен модуль має свій певний набір функцій. В таблиці 2.14 наведено загальну модель вимог, а в таблиці 2.15 наведений опис функціональних вимог до програмного забезпечення. Матрицю трасування вимог можна побачити на таблиці 2.16.

Таблиця 2.14 – Загальна модель вимог

№	Назва	ID Вимоги	Пріоритети	Ризики
1	Перегляд сторінки привітання	FR-1	Низький	Низькі
2	Реєстрація через імейл	FR-2	Високий	Високі
2.1	Реєстрація через Google OAuth 2.0	FR-3	Високий	Високі
2.2	Авторизація користувача	FR-4	Високий	Високі
2.3	Логін через імейл	FR-5	Середній	Середні
2.4	Логін через Google OAuth 2.0	FR-6	Середній	Середні
3	Відновлення паролю	FR-7	Високий	Високі

3.1	Запит на відновлення паролю	FR-8	Високий	Високі
3.2	Встановлення нового паролю	FR-9	Високий	Високі
4	Перегляд деталей профілю	FR-10	Середній	Середні
4.1	Зміна деталей профілю	FR-11	Середній	Середні
4.2	Зміна аватарки профілю	FR-12	Середній	Низькі
5	Перегляд списку дисциплін	FR-13	Середній	Низькі
5.1	Додавання дисциплін до списку	FR-14	Середній	Низькі
5.2	Видалення дисципліни зі списку	FR-15	Середній	Низькі
6	Перегляд дисципліни	FR-16	Середній	Середні
6.1	Перегляд списку завдань	FR-17	Середній	Низькі
6.2	Управління списком завдань	FR-18	Високий	Високі
6.3	Створення нового завдання	FR-19	Високий	Високі
6.4	Видалення завдання	FR-20	Середній	Середні
6.5	Створення дедлайнів для завдання	FR-21	Високий	Середні
6.6	Створення нового тестового завдання	FR-22	Високий	Середні
6.7	Створення варіантів для тесту	FR-23	Високий	Високі
6.8	Створення варіантів відповіді для тестів	FR-24	Середній	Низькі
6.9	Збереження завдання	FR-25	Середній	Низькі
7	Редагування тестів	FR-26	Високий	Середні
7.1	Відкриття завдання	FR-27	Середній	Низькі
7.2	Перегляд тестів	FR-28	Середній	Низькі

8	Перегляд списку студентів на предметі	FR-29	Середній	Низькі
8.1	Додавання студентів на предмет	FR-30	Середній	Середні
9	Проходження завдання	FR-31	Високий	Середні
9.1	Збереження відповідей	FR-32	Високий	Середні
10	Розрахунок оцінки за завдання	FR-33	Високий	Високі
10.1	Перегляд поточних оцінок	FR-34	Середній	Середні
10.2	Перегляд поточних оцінок студентів	FR-35	Середній	Низькі
11	Запрошення користувачів у систему	FR-36	Високий	Середні

Таблиця 2.15 – Перелік функціональних вимог

Назва	Опис
FR-1	Перегляд сторінки привітання. Неавторизований користувач бачить веб-сторінку привітання. На ній можна обрати можливість зареєструватися або авторизуватися.
FR-2	Реєстрація через імейл. Користувач заповнює форму реєстрації, вказуючи імейл і пароль, які перевіряються на відповідність вимогам.
FR-3	Реєстрація через Google OAuth 2.0. Користувач має можливість зареєструватися, використовуючи свій акаунт Google.
FR-4	Авторизація користувача. Користувач вводить імейл та пароль для входу в систему, які перевіряються на правильність.
FR-5	Логін через імейл. Користувач може увійти в систему, використовуючи імейл і пароль.
FR-6	Логін через Google OAuth 2.0. Користувач може увійти в систему за допомогою свого акаунту Google.

FR-7	Відновлення паролю. Користувач може відновити доступ до облікового запису через імейл.
FR-8	Запит на відновлення паролю. Користувач вводить імейл для отримання листа з посиланням на зміну паролю.
FR-9	Встановлення нового паролю. Користувач переходить за посиланням із листа та встановлює новий пароль.
FR-10	Перегляд деталей профілю. Користувач може переглядати інформацію свого профілю, зокрема ім'я, аватарку та інші дані.
FR-11	Зміна деталей профілю. Користувач може змінювати свої персональні дані, такі як ім'я чи контактну інформацію.
FR-12	Зміна аватарки профілю. Користувач може завантажувати та змінювати свою аватарку.
FR-13	Перегляд списку дисциплін. Користувач бачить перелік дисциплін, до яких він має доступ.
FR-14	Додавання дисциплін до списку. Користувач може додавати нові дисципліни до свого списку.
FR-15	Видалення дисципліни зі списку. Користувач може видаляти дисципліни зі свого списку.
FR-16	Перегляд дисципліни. Користувач може переглядати деталі обраної дисципліни.
FR-17	Перегляд списку завдань. Користувач може переглядати перелік завдань для обраної дисципліни.
FR-18	Управління списком завдань. Користувач може додавати, змінювати та видаляти завдання у списку.
FR-19	Створення нового завдання. Користувач може створювати нове завдання, задаючи його параметри.
FR-20	Видалення завдання. Користувач може видаляти створені завдання.
FR-21	Створення дедлайнів для завдання. Користувач може встановлювати дедлайни для виконання завдань.

FR-22	Створення нового тестового завдання. Користувач може створювати тестові завдання для студентів.
FR-23	Створення варіантів для тесту. Користувач задає варіанти відповідей для тестових завдань.
FR-24	Створення варіантів відповіді для тестів. Користувач може додавати можливі відповіді до тестів.
FR-25	Збереження завдання. Користувач може зберігати створене завдання для подальшого редагування або виконання.
FR-26	Редагування тестів. Користувач може змінювати вміст і параметри тестів.
FR-27	Відкриття завдання. Користувач може переглянути завдання в окремому вікні або вкладці.
FR-28	Перегляд тестів. Користувач може переглядати створені тести для дисципліни.
FR-29	Перегляд списку студентів на предметі. Користувач бачить список студентів, які підписані на дисципліну.
FR-30	Додавання студентів на предмет. Користувач може додавати студентів до дисципліни вручну або через імпорт.
FR-31	Проходження завдання. Студенти можуть виконувати завдання в системі.
FR-32	Збереження відповідей. Студенти можуть зберігати свої відповіді під час виконання завдання.
FR-33	Розрахунок оцінки за завдання. Система автоматично обчислює оцінку за виконане завдання.
FR-34	Перегляд поточних оцінок. Користувач може переглядати свої оцінки за дисципліни.
FR-35	Перегляд поточних оцінок студентів. Викладач може переглядати оцінки студентів у дисципліні.

FR-36	Запрошення користувачів у систему. Користувач може запрошувати інших користувачів через імейл.
FR-37	Відправка імейлів про запрошення. Система автоматично надсилає запрошення на вказані адреси.

	FR-1	FR-2	FR-3	FR-4	FR-5	FR-6	FR-7	FR-8	FR-9	FR-10	FR-11	FR-12	FR-13	FR-14	FR-15	FR-16	FR-17
UC-01	+	+	+	+													
UC-02				+	+	+											
UC-03							+	+	+								
UC-04																	
UC-05												+	+	+	+	+	+
UC-06										+	+						
UC-07																	
UC-08																	
UC-09																	
UC-10																	
UC-11																	
UC-12																	

Рисунок 2.2 – Матриця трасування вимог

Розроблення нефункціональних вимог

Нефункціональні вимоги визначають ключові властивості, які забезпечують стабільну, ефективну та зручну роботу веб-застосунку. Вони охоплюють аспекти продуктивності, безпеки, масштабованості, зручності використання, надійності та обслуговування.

Система повинна оперативно відповідати на запити користувачів. Час реакції інтерфейсу на будь-яку дію не повинен перевищувати 1 секунду. Обробка основних операцій, таких як збереження даних чи завантаження контенту, має бути виконана без помітних затримок. Система повинна ефективно масштабуватися для підтримки стабільної роботи під час збільшення навантаження.

Веб-застосунок має забезпечувати надійний захист даних користувачів.

Конфіденційна інформація повинна передаватися через захищені протоколи (SSL/TLS). Слід впроваджувати регулярні оновлення безпеки та застосовувати сучасні підходи до управління доступом, щоб запобігати несанкціонованим діям.

Застосунок має бути доступним для користувачів без перерв. Висока доступність (понад 99.5%) досягається завдяки використанню резервних серверів і механізмів відновлення після збоїв. У разі проблем у певній частині системи інші модулі повинні продовжувати працювати.

Інтерфейс веб-застосунку повинен бути зрозумілим і адаптованим для різних пристроїв, включаючи мобільні. Дії користувачів, такі як пошук інформації чи виконання основних операцій, мають бути інтуїтивно зрозумілими та займати мінімум часу.

Застосунок повинен підтримувати додавання нових ресурсів (серверів або функціональних модулів) при збільшенні кількості користувачів або даних. Це дозволить стабільно працювати навіть під час пікових навантажень.

Веб-застосунок має бути доступним для роботи у всіх популярних браузерях (Chrome, Firefox, Safari, Edge) і підтримувати основні операційні системи (Windows, macOS, Linux).

Система також повинна бути легкою в адмініструванні. Оновлення та виправлення помилок повинні впроваджуватися швидко. Для адміністраторів має бути передбачений зручний інтерфейс для управління функціоналом, а також механізми моніторингу стану системи для оперативного вирішення можливих проблем.

Висновки до розділу

У цьому розділі було детально розглянуто вимоги до програмного забезпечення платформи для управління освітніми дисциплінами, включаючи функціональні та нефункціональні аспекти. Функціональні вимоги охоплюють ключові процеси, такі як реєстрація та авторизація користувачів, управління дисциплінами, створення та оцінювання завдань, а також нотифікація користувачів. Для кожного процесу було створено діаграми варіантів використання, які ілюструють послідовність дій та взаємодію користувачів із системою. Описані варіанти використання містять чітке визначення цілей, акторів, умов виконання, основного потоку подій, можливих розширень і пост-умов, що забезпечує повне розуміння функціональності платформи.

Нефункціональні вимоги охоплюють ключові характеристики системи, які гарантують її ефективну та стабільну роботу. Це включає високу продуктивність, безпеку даних, надійність, зручність використання, масштабованість та сумісність із різними пристроями й браузерами. Платформа повинна забезпечувати швидку обробку запитів, захист персональних даних, стабільну роботу під навантаженням, а також підтримувати інтуїтивно зрозумілий інтерфейс для користувачів із різними ролями.

На основі аналізу вимог і результатів трасування було сформовано технічне завдання, яке стане основою для розробки програмного забезпечення. Це забезпечує створення платформи, яка буде відповідати потребам користувачів, а також здатна масштабуватися відповідно до зростаючих вимог і навантаження.

КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Архітектура програмного забезпечення

Архітектура платформи для управління освітніми дисциплінами побудована на клієнт-серверній архітектурі. Фронтенд забезпечує зручний користувацький інтерфейс для студентів, викладачів та адміністраторів, тоді як бекенд відповідає за бізнес логіку нашої платформи. Платформа, також, інтегрується із зовнішніми сервісами, такими як система електронної пошти для надсилання сповіщень про запрошення або відновлення доступу, а також із зовнішнім API для аутентифікації через Google OAuth 2.0. Зберігання всієї необхідної інформації відбувається в базі даних. Така архітектура забезпечує масштабованість, безпеку та гнучкість, що є ключовими для ефективного функціонування освітньої платформи.

На першому рівні діаграми C4 (рис. 3.1) всі компоненти системи представлені як загальні елементи, що взаємодіють на високому рівні. У цьому контексті центральним елементом є платформа для управління освітніми дисциплінами StudyHub. Основні користувачі системи — це студенти, викладачі та адміністратори.

Студенти використовують платформу для перегляду навчальних матеріалів, виконання завдань і отримання результатів. Викладачі створюють навчальні дисципліни, перевіряють роботи та виставляють оцінки. Адміністратори керують користувачами, дисциплінами та іншими адміністративними процесами.

Окрім цього, платформа взаємодіє із зовнішніми системами, такими як електронна пошта для сповіщень і зовнішній API для аутентифікації через Google OAuth 2.0. Вся інформація про користувачів, дисципліни та результати зберігається в базі даних. Ця діаграма дає загальне уявлення про основні взаємодії в системі, не заглиблюючись у деталі кожного компонента.

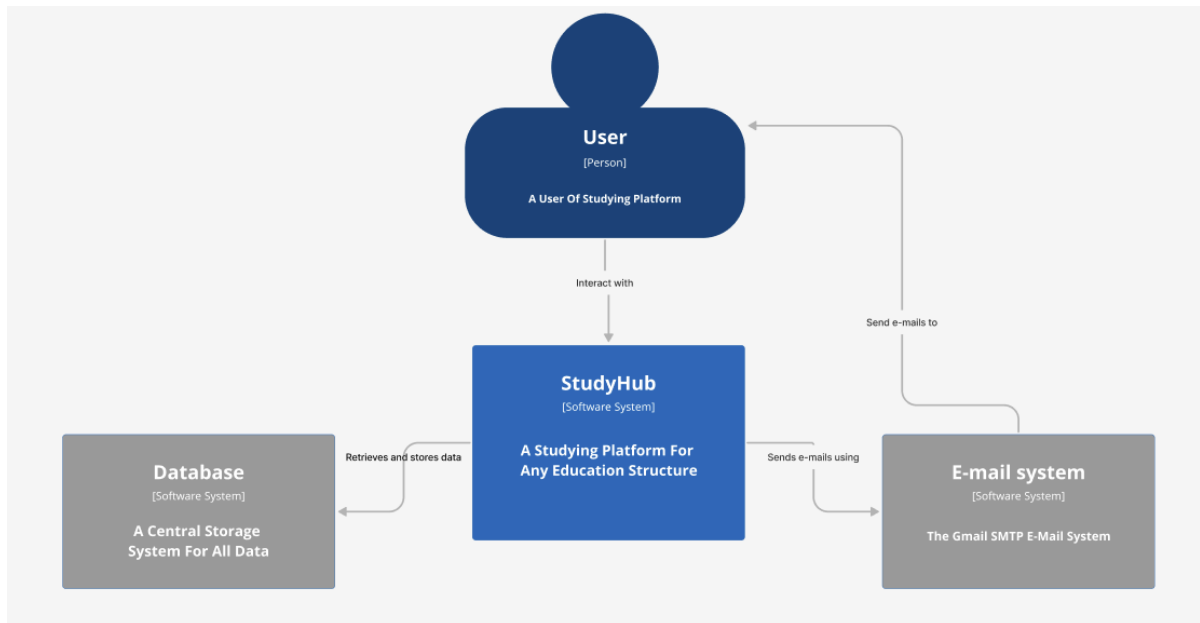


Рисунок 3.1 – Перший рівень діаграми C4

Діаграма другого рівня C4 (рис. 3.2) надає глибше розуміння архітектури платформи для управління освітніми дисциплінами, детально зображуючи ключові компоненти та їхні взаємодії. Окрім вже згаданих базових елементів, таких як фронтенд, бекенд, база даних, електронна пошта та зовнішній API для аутентифікації, варто також зазначити кілька важливих аспектів:

- **Frontend:** Відповідає за можливість інтеграції з користувачами. Він забезпечує відображення та взаємодією з інформацією та прийняттям даних, які передаються до бекенду через API.
- **Backend:** На основі мови програмування .NET реалізує бізнес-логіку платформи, включаючи управління дисциплінами, завданнями, оцінками та користувачами. Також она з його функцій – валідація даних, та при виникненні певних проблем правильно оброблювати їх. Він також взаємодіє з базою даних для зберігання та отримання інформації через EntityFramework та LINQ для побудови запитів.
- **Database:** База даних для зберігання та читання з неї інформації на базі SQL Server. Використовується бекендом через EntityFramework який транслює запити до нашої бази даних

- Email system: Використовує електронну пошту для надсилання повідомлень про запрошення до системи та відновлення паролю. Усі дії виконуються за допомогою SMTP серверу
- Google OAuth 2.0: Стороння API, яка дозволяє користувачам авторизуватися на платформі за допомогою облікових записів Google, забезпечуючи зручність і безпеку доступу.

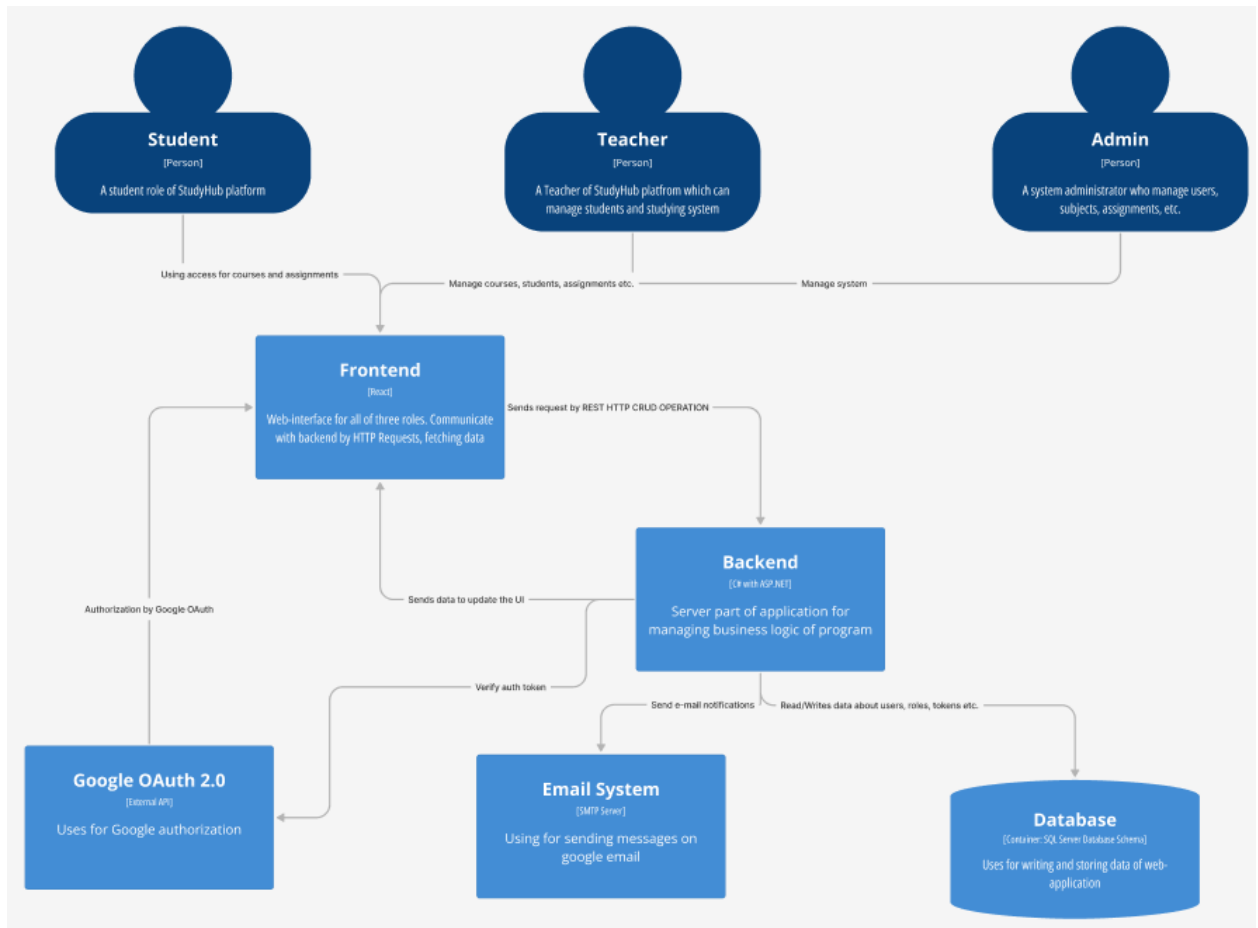


Рисунок 3.2 – Другий рівень діаграми С4

Діаграма третього рівня С4 (рис. 3.3) деталізує внутрішню структуру та взаємодію компонентів у бекенді інтелектуальної платформи управління освітніми дисциплінами. Вона описує основні сервіси, контролери та їхні зв'язки, що забезпечують функціонування всієї системи:

Контролери (Controllers):

- **SubjectsController:** Відповідає за обробку HTTP-запитів, пов'язаних із дисциплінами, включаючи отримання списку дисциплін та взаємодію з клієнтом.
- **UsersController:** Обробляє запити, що стосуються взаємодії з користувачами, включаючи зміну профільної інформації, запрошення користувачів тощо.
- **AuthController:** Відповідає за авторизацію та автентифікацію користувачів, зокрема створення токенів та перевірку облікових даних.
- **GoogleAuthController:** Спеціалізується на обробці запитів, пов'язаних із Google OAuth, для авторизації через Google.

Сервіси (Services):

- **SubjectsService:** Реалізує бізнес-логіку, пов'язану з освітніми дисциплінами, наприклад, обробку даних для клієнтської частини.
- **UsersService:** Виконує бізнес-логіку, пов'язану із взаємодією користувачів із системою, включаючи управління запрошеннями та оновлення інформації.
- **AuthService:** Реалізує функціонал, пов'язаний із авторизацією, наприклад, перевіркою паролів, створенням токенів та скиданням паролів.
- **GoogleAuthService:** Сервіс-обгортка для взаємодії з Google API через HTTP-запити для авторизації користувачів за допомогою Google OAuth.

Інфраструктурні компоненти:

- **DbContext:** Забезпечує безпечний та прямий доступ до бази даних для зчитування та запису даних платформи.
- **Repository<T>:** Генеричний репозиторій для взаємодії з базою даних, що дозволяє обробляти запити для будь-якого типу даних через DbContext.

Зовнішні системи:

- **Google OAuth 2.0:** Використовується для автентифікації користувачів через Google.

- E-mail System: Система SMTP, що забезпечує надсилання електронних листів, наприклад, для підтвердження реєстрації чи відновлення пароля.
- Database: Центральне сховище даних для зберігання інформації про користувачів, дисципліни, авторизацію тощо.

Діаграма показує, як кожен компонент взаємодіє між собою, включаючи передачу даних між сервісами та зовнішніми системами. Це дозволяє зрозуміти, як платформа реалізує основні функції, такі як авторизація, управління дисциплінами та взаємодія з користувачами, на рівні компонентів.

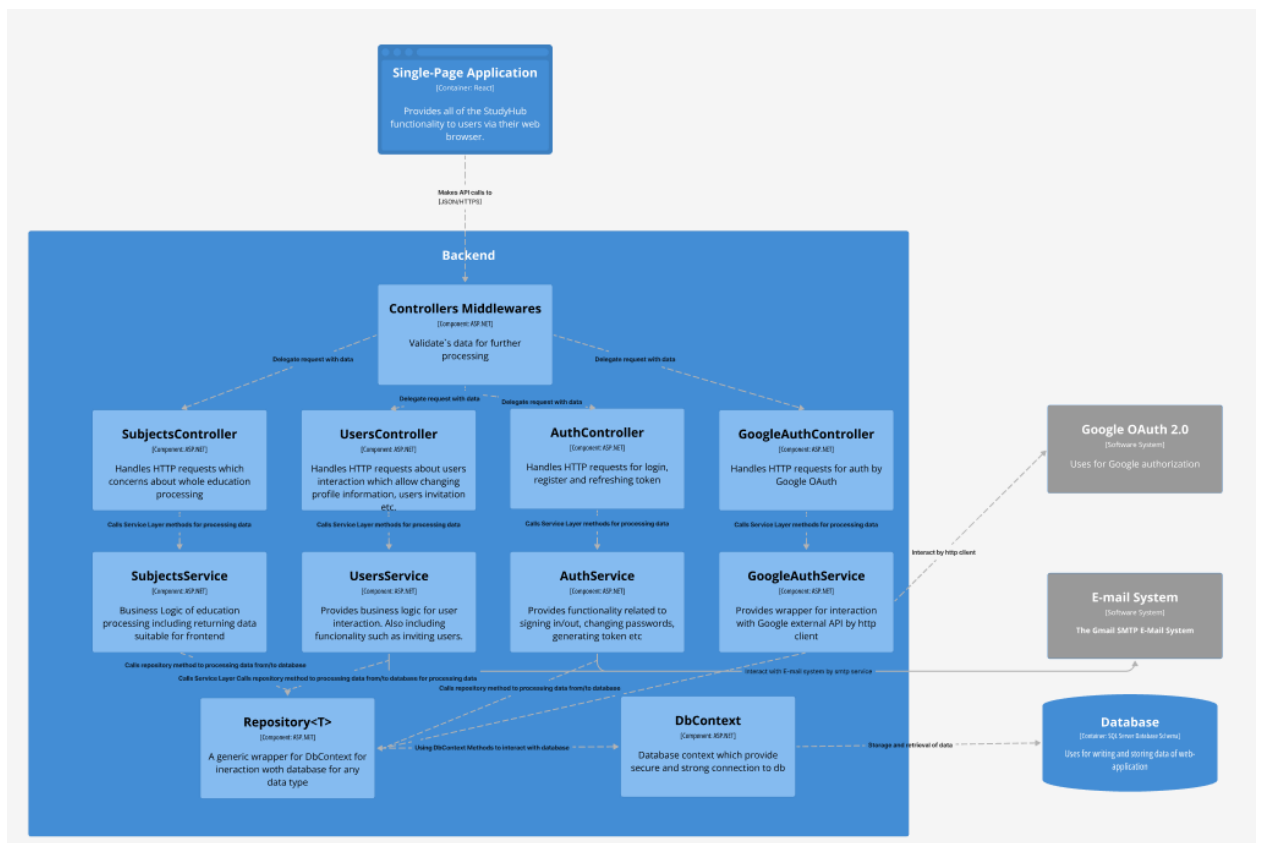


Рисунок 3.3 – Третій рівень діаграми С4

Діаграма послідовностей для процесу авторизації через Google OAuth (рис. 3.4) ілюструє основні кроки, які виконуються для надання користувачеві доступу до системи з використанням Google як зовнішнього провайдера аутентифікації. Ця послідовність демонструє взаємодію між користувачем, клієнтським інтерфейсом, серверною частиною, Google OAuth API та базою даних.

- Ініціація авторизації: Користувач натискає кнопку "Sign in with Google" на інтерфейсі. Це тригерує запит до серверної частини для початку процесу аутентифікації.
- Перенаправлення до Google OAuth: Сервер генерує спеціальний URL для авторизації та перенаправляє користувача до Google OAuth, передаючи необхідні параметри (клієнтський ID, URL для зворотного виклику тощо).
- Авторизація на стороні Google: Користувач вводить свої облікові дані Google (якщо вони ще не введені) і дозволяє доступ до певних даних свого профілю.
- Отримання коду авторизації: Google перенаправляє користувача назад до системи із кодом авторизації, який є короткостроковим маркером для отримання токена доступу.
- Обмін коду на токен доступу: Серверна частина відправляє код авторизації до Google OAuth API для отримання токенів (токена доступу та, опціонально, оновлювального токена).
- Отримання інформації про користувача: За допомогою отриманого токена сервер викликає API Google для отримання даних користувача, таких як електронна пошта та ім'я.
- Збереження даних у базі: Сервер перевіряє, чи користувач уже існує в базі даних. Якщо ні, створюється новий запис. У випадку існування облікового запису сервер оновлює відповідну інформацію.
- Видача токена для клієнта: Сервер генерує JWT (JSON Web Token) для користувача, який буде використовуватися для авторизації в подальших запитах, і передає його на клієнт.
- Доступ до системи: Користувач отримує доступ до системи та може виконувати дії, доступні для його ролі.

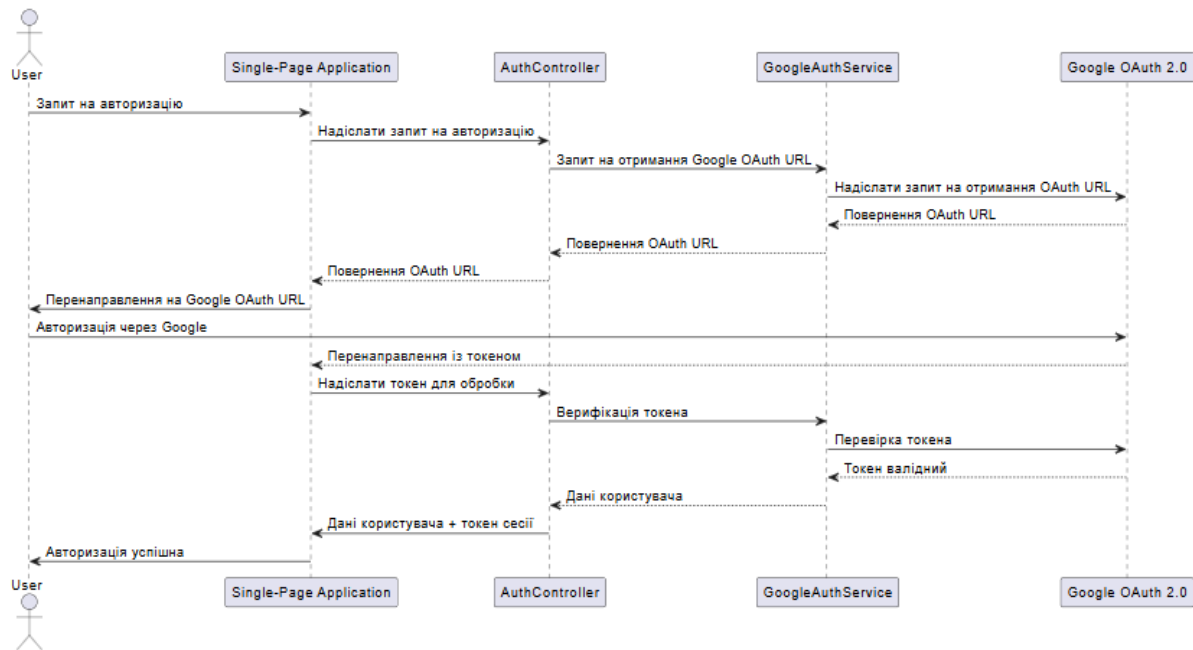


Рисунок 3.4 – Діаграма послідовностей

Обґрунтування вибору засобів розробки

У цьому розділі оглянемо та проаналізуємо допоміжні програмні засоби та засоби розробки, що використовуються у даному проєкті. Для розробки було використано бекенд-фреймворк ASP.NET з ORM Entity Framework, а також React.ts на фронтенді із додатковими бібліотеками MUI і React Query. Для зберігання даних використовується реляційна база даних SQL Server.

ASP.NET – сучасний і продуктивний бекенд-фреймворк який забезпечує високу продуктивність, масштабованість та надійність веб-застосунків. Вибір пав саме на нього і мову програмування .NET, оскільки завдяки вистроєній ієрархії та абстракції – наша система буде зрозуміла, та легко доповнювана. Також ASP.NET має інтеграцію з такими потужними бібліотеками як EntityFramework, Hangfire, AutoMapper, FluentValidation та інші

EntityFramework – це ORM (Object-Relational Mapping) для зручної взаємодії з ДБ. Основними плюсами цього ORM є те, що він дозволяє працювати с записами на рівні абстракції використовуючи LINQ(Language Integrated Query) для будування запитів та можливість ефективного та безпечного будування міграцій у разі розширення БД.

SQL Server – СУБД, який пропонує стабільну й ефективну роботу з реляційними базами даних. Наш вибір пав на реляційні бази даних, адже наша система має на собі структуру чіткої, структурованої системи з чітко визначеними значеннями, тому SQL – ідеальний кандидат.

Hangfire – бібліотека, яка дозволяє реалізувати бізнес логіку пов’язана з фоновими задачами (Background Jobs). Він дозволяє створювати, запускати та моніторити фонові довготривалі задачі без необхідності використання зовнішніх планувальників.

React.ts – бібліотека для розробки клієнтської частини програми на основі мови програмування TypeScript. React дозволяє створювати динамічні, інтерфейси, що легко оновлюються і масштабуються завдяки компонентному підходу. Використання TypeScript додає строгий контроль типів, знижуючи кількість помилок на етапі розробки і підвищуючи якість коду. React має активну спільноту, численні бібліотеки й інструменти для прискорення розробки, що робить його ідеальним вибором для сучасних веб-додатків.

MUI(Material UI) – бібліотека компонентів для React яка реалізує принципи Material Design від Google. MUI пропонує набір готових, стилізованих і адаптивних компонентів, таких як кнопки, поля введення, таблиці та модальні вікна, що значно скорочує час розробки.

Конструювання програмного забезпечення

Для функціонування застосунку була побудована база даних яка відображена на рисунку 3.5. Ця база даних призначена для зберігання даних

про юзерів, їх ролями, токенами, користувачів які були запрошені до системи, навчальні дисципліни, їх завдання з тестами, оцінками тощо.

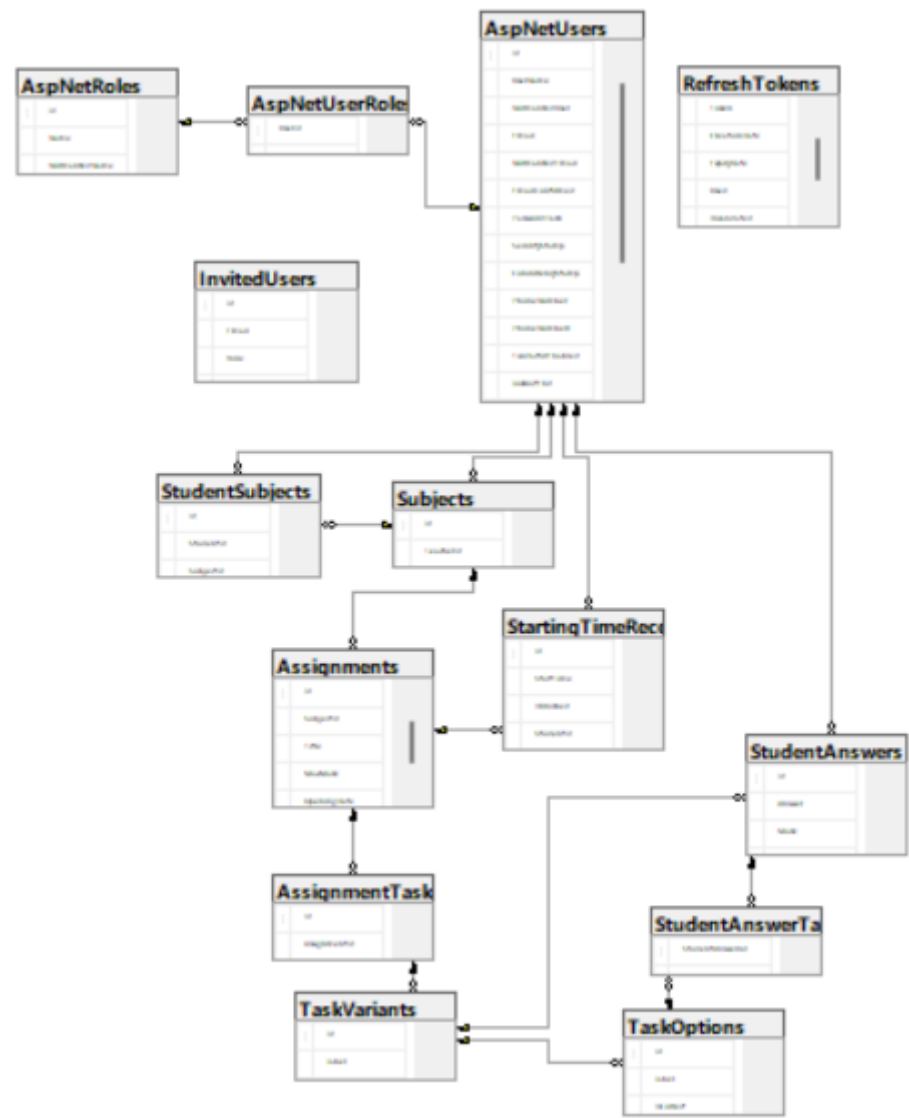


Рисунок 3.5 – ER діаграма сутностей

Таблиця 3.1 – Опис таблиці AspNetUsers

Назва поля	Тип даних	Опис
Id	GUID(PK)	Унікальний ідентифікатор користувача
UserName	NVARCHAR	Логін користувача

NormalizedUserName	NVARCHAR	Нормалізований логін
Email	NVARCHAR	Email користувача
NormalizedEmail	NVARCHAR	Нормалізований Email
EmailConfirmed	BOOLEAN	Підтвердження Email
PasswordHash	NVARCHAR	Хеш пароля
SecurityStamp	NVARCHAR	Відмітка безпеки
ConcurrencyStamp	NVARCHAR	Поле для контролю версій
PhoneNumber	NVARCHAR	Номер телефону
PhoneNumberConfirmed	BOOLEAN	Чи підтверджено телефон
TwoFactorEnabled	BOOLEAN	Чи ввімкнено двофакторну автентифікацію
LockoutEnd	DATETIME	Кінець періоду блокування
LockoutEnabled	BOOLEAN	Чи доступне блокування
AccessFailedCount	INT	Кількість невдалих спроб входу
Course	NVARCHAR	Курс навчання

FullName	NVARCHAR	Повне ім'я
[Group]	NVARCHAR	Група
Telegram	NVARCHAR	Telegram-аккаунт
Faculty	NVARCHAR	Факультет

Таблиця 3.2 – Опис таблиці AspNetUserRole

Назва поля	Тип даних	Опис
UserId	GUID(FK)	Ідентифікатор користувача
RoleId	GUID(FK)	Ідентифікатор ролі

Таблиця 3.3 – Опис таблиці AspNetRoles

Назва поля	Тип даних	Опис
Id	GUID(PK)	Унікальний ідентифікатор ролі
Name	NVARCHAR	Назва ролі
NormalizedName	NVARCHAR	Нормалізована назва ролі
ConcurrencyStamp	NVARCHAR	Поле для контролю версій

Таблиця 3.4 – Опис таблиці RefreshTokens

Назва поля	Тип даних	Опис
Token	NVARCHAR	Значення токена
CreationDate	DATETIME	Дата створення токена
ExpiryDate	DATETIME	Дата закінчення дії токена
Used	BOOLEAN	Чи використовувався токен
Invalidated	BOOLEAN	Чи недійсний токен
Id	GUID(PK)	Унікальний ідентифікатор токена
UserId	GUID(FK)	Ідентифікатор користувача

Таблиця 3.5 – Опис таблиці InvitedUsers

Назва поля	Тип даних	Опис
Id	GUID(PK)	Унікальний ідентифікатор запрошення
Email	NVARCHAR	Email запрошеного користувача
Role	NVARCHAR	Роль запрошеного
Token	NVARCHAR	Унікальний токен запрошення

CreatedAt	DATETIME	Дата створення запрошення
-----------	----------	------------------------------

Таблиця 3.6 – Опис таблиці StudentSubjects

Назва поля	Тип даних	Опис
Id	GUID(PK)	Унікальний ідентифікатор запису
StudentId	GUID(FK)	Ідентифікатор студента
SubjectId	GUID(FK)	Ідентифікатор предмету
Mark	INT	Оцінка студента за предметом

Таблиця 3.7 – Опис таблиці Subjects

Назва поля	Тип даних	Опис
Id	GUID(PK)	Унікальний ідентифікатор
TeacherId	GUID(FK)	Ідентифікатор викладача
Title	NVARCHAR	Назва предмету

Таблиця 3.8 – Опис таблиці Assignments

Назва поля	Тип даних	Опис
Id	GUID(PK)	Унікальний ідентифікатор завдання

SubjectId	GUID(FK)	Ідентифікатор предмету
Title	NVARCHAR	Назва завдання
MaxMark	INT	Максимальна оцінка
OpeningDate	DATETIME	Дата відкриття завдання
ClosingDate	DATETIME	Дата закриття завдання
Duration	INT	Тривалість виконання (хвилини)

Таблиця 3.9 – Опис таблиці StartingTimeRecords

Назва поля	Тип даних	Опис
Id	GUID(PK)	Унікальний ідентифікатор запису
StartTime	DATETIME	Час початку виконання завдання
IsFinished	BOOLEAN	Чи завершено завдання
StudentId	GUID(FK)	Ідентифікатор студента
AssignmentId	GUID(FK)	Ідентифікатор завдання
Mark	DECIMAL	Оцінка за завдання

Таблиця 3.10 – Опис таблиці StudentAnswers

Назва поля	Тип даних	Опис
Id	GUID(PK)	Унікальний ідентифікатор

Answer	NVARCHAR	Відповідь студента
Mark	INT	Оцінка
TaskVariantId	GUID(FK)	Ідентифікатор завдання
StudentId	GUID(FK)	Ідентифікатор студента

Таблиця 3.11 – Опис таблиці StudentAnswerTaskOption

Назва поля	Тип даних	Опис
StudentAnswersId	GUID(FK)	Ідентифікатор відповіді
TaskOptionsId	GUID(FK)	Ідентифікатор опції

Таблиця 3.12 – Опис таблиці TaskOptions

Назва поля	Тип даних	Опис
Id	GUID(PK)	Унікальний ідентифікатор
Label	NVARCHAR	Текст опції
IsCorrect	BOOLEAN	Чи правильна опція
TaskVariantId	GUID(FK)	Ідентифікатор завдання

Таблиця 3.13 – Опис таблиці TaskVariants

Назва поля	Тип даних	Опис
Id	GUID(PK)	Унікальний ідентифікатор
Label	NVARCHAR	Опис варіанту завдання
AssignmentTaskId	GUID(FK)	Ідентифікатор завдання

Опис утиліт, бібліотек та іншого стороннього програмного забезпечення, що використовується у розробці наведено в таблиці 3.22.

Таблиця 3.14– Опис утиліт

№ п/п	Назва утиліти	Опис застосування
1	Visual studio	Головне середовище розробки програмного забезпечення серверної частини курсової роботи на мові програмування C#.
2	Visual studio code	Головне середовище розробки програмного забезпечення клієнтської частини курсової роботи на мові програмування TypeScript.
3	DataGrip	Програмне забезпечення яке надає легкий графічний інтерфейс для доступу до бази даних.

4	Postman	Програмне забезпечення необхідне для тестування rest запитів. Використовувалось для тестування API інтерфейсів, та клієнтських запитів.
---	---------	---

Тексти програмного коду наведені в окремому документі «Текст програми».

Аналіз безпеки даних

У застосунку для забезпечення надійного захисту даних користувачів впроваджено кілька ключових механізмів безпеки, спрямованих на захист конфіденційної інформації, запобігання несанкціонованому доступу та протидію основним загрозам.

Для аутентифікації та авторизації використовується JSON Web Tokens (JWT). Реалізовано два типи токенів: Access-токени, які мають короткий термін дії, і Refresh-токени, що використовуються для оновлення доступу без повторної аутентифікації. Токени підписуються за допомогою алгоритму HMAC SHA256, що гарантує їхню цілісність і достовірність. Всі токени передаються через HTTPS, щоб виключити можливість їхнього перехоплення. Access-токени мають обмежений час дії, що мінімізує ризик зловживань у разі їхньої компрометації, а Refresh-токени зберігаються у захищених HTTP-only cookie.

Для зберігання паролів користувачів використовується ASP.NET Core Identity із застосуванням UserManager. Паролі ніколи не зберігаються у відкритому вигляді, натомість для їхнього зберігання використовується хешування за допомогою алгоритму PBKDF2. Додатково для кожного пароля додається унікальна сіль, що запобігає використанню попередньо обчислених хешів (rainbow tables). Такий підхід у поєднанні з достатньою кількістю ітерацій забезпечує стійкість до атак методом перебору. Крім того, застосунок

підтримує політику складних паролів, що вимагає мінімальної довжини та використання спеціальних символів, цифр і великих літер.

Для взаємодії з базою даних застосовується ORM Entity Framework Core, який автоматично захищає від SQL-ін'єкцій завдяки використанню параметризованих запитів. Запити формуються через LINQ, що дозволяє уникнути впровадження шкідливого коду. Навіть у разі використання сирих SQL-запитів усі значення автоматично екрануються за допомогою методів FromSqlInterpolated або ExecuteSqlInterpolated. Додатково реалізовано валідацію вхідних даних на стороні сервера через middleware, щоб забезпечити їхню відповідність очікуваному формату.

Усі дані передаються лише через захищене HTTPS-з'єднання, а підключення до бази даних налаштовано за принципом найменших привілеїв. Це означає, що обліковий запис бази даних має доступ лише до необхідних таблиць і операцій. Також впроваджено логування та моніторинг підозрілої активності для швидкого реагування на потенційні загрози.

Висновки до розділу

У межах цієї курсової роботи було розроблено архітектуру та основні компоненти веб-застосунку для проведення навчальних курсів, які забезпечують ефективне управління освітнім процесом. Було застосовано модульний підхід, що дозволяє інтегрувати фронтенд і бекенд у єдину екосистему, забезпечуючи зручну взаємодію для студентів, викладачів і адміністраторів платформи. Детально описано ключові компоненти системи, включаючи модулі для аутентифікації, управління курсами, розкладу занять, перевірки завдань тощо.

Для реалізації застосунку обрано сучасні технології, що забезпечують продуктивність, масштабованість і безпеку. На бекенді використано ASP.NET Core, а на фронтенді — React із застосуванням бібліотек MUI. База даних побудована з використанням Entity Framework Core та SQL Server, що

забезпечує ефективну обробку та збереження даних. Особливу увагу приділено захисту даних: аутентифікація здійснюється через JWT-токени (як аксес, так і рефреш), паролі користувачів надійно хешуються через UserManager, а запити до бази даних захищені від SQL-ін'єкцій завдяки використанню ORM.

Розроблена система відповідає сучасним вимогам зручності, безпеки та продуктивності. Реалізовані функціональні можливості дозволяють студентам і викладачам ефективно взаємодіяти в межах навчального процесу: реєстрація, доступ до навчальних матеріалів, організація занять, управління завданнями та отримання сповіщень реалізовані у зручному та інтуїтивному інтерфейсі.

Запропонований веб-застосунок є гнучкою та надійною платформою для організації навчальних курсів. У майбутньому передбачається інтеграція нових функцій, таких як автоматизація оцінювання, вдосконалення системи рекомендацій курсів і розширення можливостей аналітики навчального процесу, що дозволить платформі адаптуватися до потреб різних навчальних закладів та освітніх програм.

АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Аналіз якості ПЗ

Метриками для оцінки якості ПЗ обрано наступні:

- Покриття функціональних вимог. Ми можемо виміряти, скільки функціональних вимог було успішно реалізовано і виконано в програмному забезпеченні у відсотках. Наша мета - досягти 95% від всіх запланованих функціональних вимог.
- Час до відмови (MTTF — Mean Time To Failure) - вимірює середній час до виникнення помилки в роботі програми. Метою є досягнення 1000 годин без відмов.
- Час відгуку системи вимірює час, який система витрачає на обробку запиту користувача, наприклад, на завантаження сторінки тесту чи рендер сторінки предмету. Це вимірюється в мілісекундах, з метою досягти середнього часу відгуку не більше ніж 20 мілісекунди.
- Кількість помилок на 1000 рядків коду (Defects per Thousand Lines of Code — D/KSLOC) вимірює якість коду з точки зору його стійкості. Це допомагає оцінити рівень стабільності коду. Метою є не більше ніж 2 помилки на 1000 рядків коду.

Опис процесів тестування

Було виконане мануальне тестування програмного забезпечення, опис відповідних тестів наведено у таблицях 4.3 – 4.12.

Таблиця 4.3 – Тест 1.1 Реєстрація користувача

Початковий стан системи	Користувач знаходиться на сторінці реєстрації
Вхідні дані	Електронна пошта, пароль, лист з запрошенням
Опис проведення тесту	Користувачу на пошту приходить лист із запрошення по якому він переходить.

	У відповідні поля на реєстрації вводяться: повне ім'я користувача, коректна електронна пошта, яка до цього не була зареєстрована в системі, пароль від 10 до 64 символів, який містить хоча б одну велику та маленьку англійські літери, одне число і один спеціальний символ, підтвердження паролю, яке співпадає з раніше введеним паролем. Після цього натискається кнопка підтвердження реєстрації.
Очікуваний результат	Реєстрація проходить успішно, користувач додається у систему і перенаправляється на сторінку дашборд.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.4 – Тест 1.2 Авторизація користувача

Початковий стан системи	Користувач знаходиться на сторінці авторизації
Вхідні дані	Електронна пошта, пароль
Опис проведення тесту	У відповідні поля вводяться: коректна електронна пошта, яка була зареєстрована раніше, і відповідний пароль. Після цього натискається кнопка авторизації.
Очікуваний результат	Авторизація проходить успішно, користувач перенаправляється на головну сторінку.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.5 – Тест 1.3 Реєстрація через Google OAuth2

Початковий стан системи	Користувач знаходиться на сторінці реєстрації
Вхідні дані	Обліковий запис Google

Опис проведення тесту	Користувач натискає кнопку "Увійти через Google". Вибирає обліковий запис, який ще не зареєстрований у системі. Після успішного входу користувач підтверджує доступ до даних.
Очікуваний результат	Реєстрація проходить успішно, користувач додається в систему та перенаправляється на головну сторінку.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.6 – Тест 1.4 Додавання студента до дисципліни через email

Початковий стан системи	Викладач знаходиться на сторінці дисципліни
Вхідні дані	Email студента
Опис проведення тесту	Викладач натискає кнопку "Додати студента", вводить email студента, натискає "Підтвердити".
Очікуваний результат	Студент додається до дисципліни, виводиться підтвердження "Студент успішно доданий".
Фактичний результат	Студент додається до дисципліни та додається до списку студентів на данній дисципліні, виводиться підтвердження "Студент успішно доданий".

Таблиця 4.7 – Тест 1.5 Запрошення студента у застосунок

Початковий стан системи	Викладач знаходиться на сторінці запрошення
Вхідні дані	Email студентів
Опис проведення тесту	Викладач вводить email усіх студентів, і натискає кнопку „Запросити„

Очікуваний результат	Студентам на пошту приходить лист з запрошенням
Фактичний результат	Студентам на пошту приходить лист з запрошенням в якому зашитий токен для доступу до реєстрації

Таблиця 4.8 – Тест 1.6 Запрошення студента у застосунок

Початковий стан системи	Викладач знаходиться на сторінці усіх дисциплін
Вхідні дані	Назва дисципліни
Опис проведення тесту	Викладач натискає на кнопку створення і вводить назву дисципліни для створення
Очікуваний результат	Дисципліна успішно створена
Фактичний результат	Дисципліна успішно створенна і Викладач перенаправляється на сторінку новоствореної дисципліни

Таблиця 4.9 – Тест 1.7 Створення тесту

Початковий стан системи	Викладач знаходиться на сторінці дисципліни
Вхідні дані	Назва завдання, дедлайн, час виконання, завдання
Опис проведення тесту	Викладач натискає на кнопку створити тест, заповнює відповідні поля і після натискання кнопки „Створити,, – переадресовується на сторінку для створення тесту. Для кожного завдання створює варіанти завдань з відповідями і на кожен натискає кнопку „Зберегти,,
Очікуваний результат	Завдання успішно створене, студенти бачать його у своєму списку завдань

Фактичний результат	Завдання створене, висвітлене у відповідній іконці „Завдання,, , і також до кожного додається компонент який відповідає за відкритість/закритість тесту
---------------------	---

Таблиця 4.10 – Тест 1.8 Старт тесту

Початковий стан системи	Студент знаходиться на сторінці дисципліни, тест відкритий для проходження
Вхідні дані	Немає
Опис проведення тесту	Студент на сторінці дисципліни вибирає завдання яке хоче пройти і переходить на нього
Очікуваний результат	Початок тесту, запуск таймеру
Фактичний результат	Система надає студенту тест з випадковими варіантами, кешує його, і запускає таймер

Таблиця 4.11 – Тест 1.9 Закінчення тесту

Початковий стан системи	Студент знаходиться на сторінці з тестом
Вхідні дані	Немає
Опис проведення тесту	Студент, обравши всі відповіді які він вважає за правильні, натискає кнопку „Закінчити тест,,
Очікуваний результат	Відповіді були записані, оцінка виведена на таблицю з загальними оцінками
Фактичний результат	Оцінка була обрахована на основі відповідей та виведена на таблицю загального результату для дисципліни. Тест стає неактивний для проходження данного користувача.

	Оцінка також виводиться на загальний журнал оцінок для викладача
--	--

Опис контрольного прикладу

Повний опис контрольного прикладу: Створення завдання викладачом

Мета тесту: Перевірити процес створення тесту, включаючи етапи створення дедлайну, компонування тесту, його публікація. Тест перевіряє всі можливі варіанти, включаючи успішні та неуспішні стани.

Початковий стан системи:

- Користувач авторизований на сайті під роллю вчителя.
- Веб-сайт відкритий на головній сторінці або сторінці дисципліни

Крок 1: Вибір дисципліни та перехід на сторінку дисципліни (рис. 4.1)

- Користувач заходить на сторінку усіх дисципліни
- Користувач обирає потрібну йому дисципліну, натискаючи на відповідну картку.

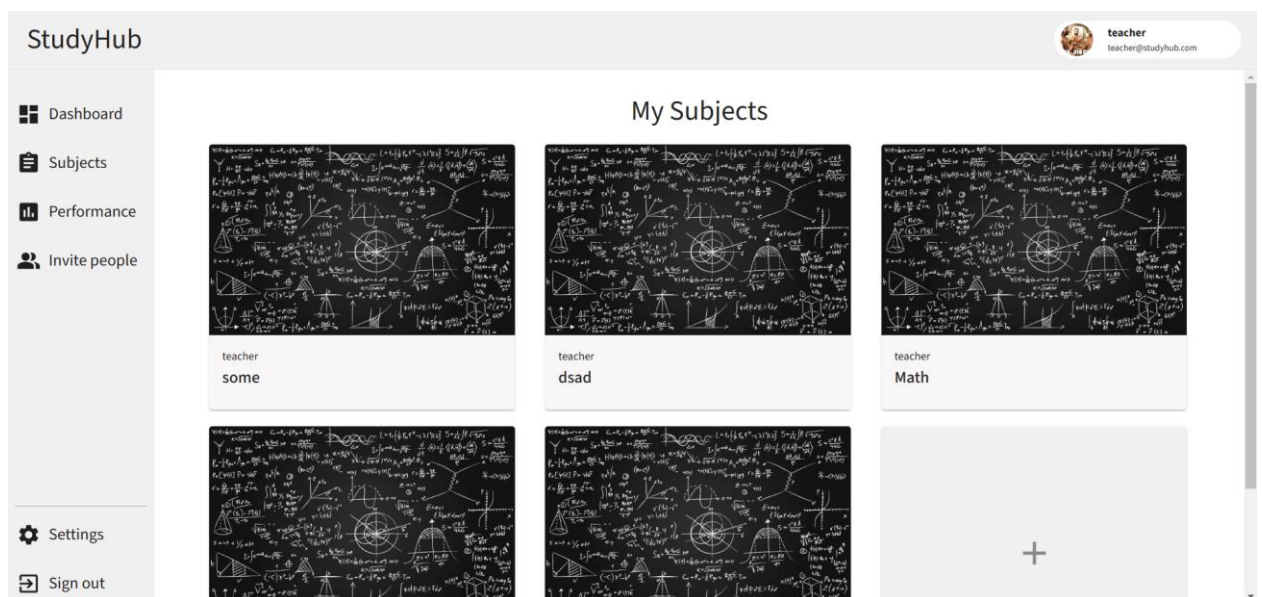


Рисунок 4.1 – крок 1

Крок 2: Перегляд поточної дисципліни, створення завдання(рис. 4.2).

- Користувач переглядає деталі дисципліни: кількість студентів на дисципліни, кількість завдань, також при бажанні може подивитися поточні оцінки студентів.

– Користувач створює нове завдання, вказує дедлайн, час виконання, кількість балів.

The screenshot shows the 'StudyHub' interface with a modal window titled 'Create a new assignment'. The modal contains the following fields and values:

- Name:** sadsqe
- Mark:** 32
- Assignment start:** 01/16/2025 12:00 AM
- Assignment end:** 01/17/2025 12:00 AM
- Duration (hh:mm:ss):** 00:30:00

At the bottom of the modal are two buttons: 'Create' and 'Back'. The background shows a sidebar with 'Dashboard', 'Subjects', 'Performance', and 'Invite people', and a main area with a list of tasks.

Рисунок 4.2 – крок 2

Крок 3: Створення першого тесту(рис. 4.3).

- Користувач створює варіанти для кожного тесту. Вказує в кожному тип тесту, питання, опції, та ставить маркер якщо опція є відповіддю
- Користувач натискає кнопку зберегти тест

The screenshot shows the 'StudyHub' interface with the 'Subjects' section. The main heading is 'Prepare your Bank of Question'. Below this is a form for creating questions:

- Question 1:** test
- Option 1:** not correct (with a 'Correct' checkbox)
- Option 2:** isCorrect (with a 'Correct' checkbox)
- Question 2:** next test

There is an 'ADD OPTION' button between the two questions. The interface also shows a sidebar with 'Dashboard', 'Subjects', 'Performance', and 'Invite people', and a top bar with the user's profile.

Рисунок 4.3 – крок 3

Крок 4: Створення наступних тестів(рис. 4.2).

- Користувач після збереження одразу створює наступний тест, або натискає кнопку створити нове питання
- Користувач робить ті ж самі дії описані в 3-ому кроці.

The screenshot shows the 'Subjects' section of the StudyHub interface. On the left is a sidebar with links to Dashboard, Subjects, Performance, and Invite people. The main area is titled 'Subjects' and 'Prepare your Bank of Question'. It features a form for creating a question. 'Question 1' has a text input with 'some question'. Below it are two options, both labeled 'right', each with a checked 'Correct' checkbox and a delete icon. There are buttons for 'ADD OPTION', '+ ADD QUESTION', 'CANCEL', and 'SAVE VARIANT'. To the right, a 'Task 1' card shows 'test' and '3'. A large pink button with a plus icon and the text 'Add New Question' is also visible.

Рисунок 4.4 – крок 4

Крок 5: Зміна існуючих тестів(рис. 4.5).

- Користувач натискає на один з створених тестів, і переглядає їхній зміст
- Користувач для зміни тесту робить ті ж самі дії описані в 3-ому кроці і натискає кнопку зберегти.

This screenshot shows the same 'Subjects' page but with updated question data. 'Question 1' now contains the text 'test'. Option 1 is 'not correct' with a checked 'Correct' checkbox, and Option 2 is 'isCorrect' with an unchecked 'Correct' checkbox. The 'ADD OPTION' and '+ ADD QUESTION' buttons are still present. On the right, the 'Task 1' card is shown, and a new 'Task 2' card appears with 'dsqa' and '3'. The pink 'Add New Question' button remains at the bottom.

Рисунок 4.5 – крок 5

Крок 6: Редагування існуючих тестів (рис. 4.6).

- Користувач повертається на сторінку дисципліни

– Користувач для редагування завдання обирає не відкритий тест, і редагує його так, як описано раніше в кроках 3-5

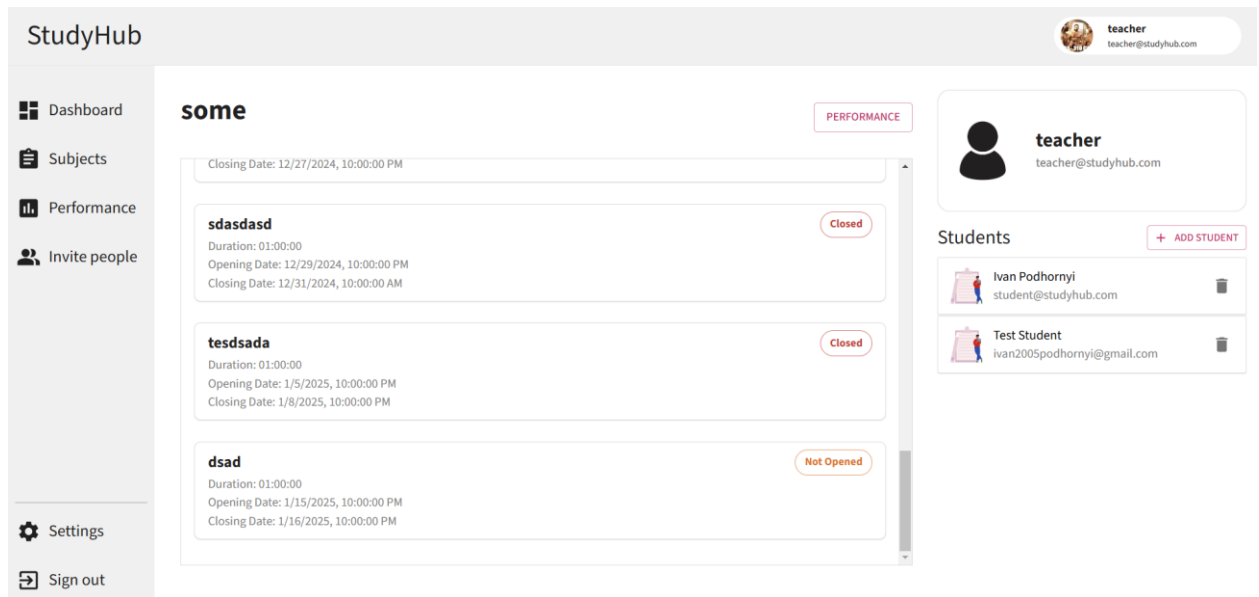


Рисунок 4.6 – крок 6

Висновки до розділу

У процесі тестування системи управління навчальним процесом було здійснено детальну перевірку її функціональних можливостей шляхом мануального тестування. Тести охопили ключові процеси, які мають значення для викладачів, студентів та адміністраторів системи, зокрема реєстрацію, авторизацію, створення та управління дисциплінами, додавання завдань, виставлення оцінок, а також можливість перегляду результатів навчання.

Результати тестування продемонстрували коректну роботу всіх основних функціональних можливостей. Система адекватно реагує на введені дані відповідно до технічних вимог. Тестування охоплювало як позитивні сценарії, так і випадки, коли вводились некоректні або неповні дані. Це дозволило перевірити стійкість та надійність програмного забезпечення у різних ситуаціях.

Особливу увагу приділили тестуванню таких важливих функцій, як реєстрація користувача, авторизація, управління дисциплінами, додавання студентів до дисциплін, створення завдань та контроль оцінювання. Система

успішно справляється із запитами, забезпечуючи зручність користування та безпеку даних. Додатково перевірено функціонал коментування із тегуванням користувачів, що значно покращує комунікацію між студентами та викладачами.

Усі проведені тести підтвердили, що система управління навчальним процесом відповідає заданим функціональним вимогам. Вона працює стабільно за умов коректного використання та демонструє здатність обробляти різні сценарії взаємодії з користувачами.

З огляду на результати тестування можна стверджувати, що програмне забезпечення готове до запуску та здатне забезпечити безперебійну роботу для всіх категорій користувачів. Система повністю відповідає функціональним вимогам і готова до використання в реальних умовах.

РОЗГОРТАННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Розгортання та супровід програмного забезпечення

Для розгортання та супровіду програмного забезпечення використовуємо локальний підхід. Для управління версіями код зберігається у GitHub, також це допоможе нам для спільної роботи для подальшого розвитку проекту.

Для розгортання, попередньо, потрібно встановити необхідне програмне забезпечення таке як: .NET SDK, Node 22.12.0 та SQL Server. Далі скопувати репозиторій з GitHub. Актуалізація файлів конфігурації відбувається у файлах appsettings.json для .NET та .env для Node. Також для локального використання потрібно в консолі серверної частини прописати команду Update-Database – це допоможе нам застосувати усі поточні міграції бази даних. Для локального запуску нам потрібно прописати дві команди – dotnet run для серверної частини і npm start – для клієнтської.

Супровід коду відбувається наступним чином:

- 1) Оновлення коду відбувається через зміни в локальній гілці, після чого створюється pull request для злиття її в основну. Варто пам'ятати, що після повноцінного розгортання develop гілка призначена для тестування функціоналу на тестовому сервері, а main – для продакшн
- 2) Оновлення бази даних відбувається за допомогою системи міграцій. Після змін в Entity проєкті нашої серверної частини, потрібно прописати команду Add-Migration – після цього оновити ДБ. Обов'язково потрібно буде оновити бази і для задеплойних гілок
- 3) Кожна нова функція має бути ретельно протестована для виявлення та виправлення можливих дефектів

Повна інструкція користувача

Для запуску серверної частини застосунку необхідно виконати наступні кроки

- Підготовка середовища

Встановлення .NET 8 та MS SQL Server

- Склонувати репозиторій застосунку

`git clone`

- Оновити базу даних та замінити в `appsettings connection_string` на свій власний

`Update-Database`

- Запустити додаток

`dotnet run`

Для перевірки чи працює бекенд частина застосунку відвідайте сайт <https://localhost:7019/swagger/index.html>

Для запуску клієнта необхідно виконати наступні кроки

- Підготовка середовища

Встановлення Node 22.12.0

- Склонувати репозиторій застосунку

`git clone`

- Запустити додаток виконавши команди

`npm install`

`npm start`

Для перевірки чи працює фронтенд частина застосунку відвідайте сайт <https://localhost:3000>

Висновки до розділу

У цьому розділі було описано процес розгортання та супроводу програмного забезпечення, розробленого із використанням стеку технологій .NET, React та MS SQL Server. Основна увага приділялася локальному підходу, який дозволяє забезпечити зручність у розробці, тестуванні та підтримці системи.

Система управління версіями GitHub була обрана як ефективний інструмент для зберігання коду, контролю змін та співпраці між розробниками. Описані інструкції забезпечують чітке розуміння процесу

підготовки середовища, налаштування конфігурацій, роботи з базою даних, а також запуску серверної та клієнтської частин.

Підхід до супроводу системи включає регулярне оновлення коду, застосування міграцій бази даних та обов'язкове тестування нових функціональних можливостей. Такий підхід сприяє підтриманню стабільності, функціональності та безперервному розвитку програмного забезпечення.

Таким чином, дотримання рекомендацій щодо розгортання та супроводу забезпечує коректну роботу системи та її готовність до подальшої експлуатації та вдосконалення.

ВИСНОВКИ

Розробка веб-застосунку для проведення навчальних курсів у межах цієї курсової роботи дозволила створити сучасну платформу, яка відповідає потребам викладачів, студентів та адміністраторів у галузі онлайн-освіти. Застосунок забезпечує зручність організації навчального процесу, автоматизацію ключових функцій та можливість взаємодії між усіма учасниками освітнього процесу. Розроблений інтерфейс дозволяє швидко знаходити навчальні матеріали, реєструватися на курси та ефективно керувати освітніми активностями, забезпечуючи позитивний досвід користувачів.

Практична значимість одержаних результатів полягає у створенні багатофункціонального веб-застосунку, який об'єднує системи реєстрації, управління курсами, інтеграції з зовнішніми сервісами та захищеної аутентифікації. Така система не лише підвищує зручність для користувачів, але й надає адміністраторам потужний інструмент для організації освітнього процесу, спрощуючи керування курсами, матеріалами та ролями користувачів.

Система пропонує багатофункціональний інтерфейс, який забезпечує різноманітні можливості для кожної ролі користувача. Ключовим є гнучкий доступ через реєстрацію або авторизацію як за допомогою імейлу та пароля, так і через Google OAuth2, що спрощує взаємодію з платформою. Для викладачів передбачено створення предметів, керування оцінками та завданнями, а також запрошення студентів і колег через імейл, що полегшує координацію навчального процесу. Студенти, у свою чергу, можуть переглядати свої оцінки та виконувати завдання, зосереджуючись на навчанні. Адміністратори мають інструменти для управління контентом і контролю якості, зокрема можливість видаляти небажані дані або запрошувати нових адміністраторів. Завдяки чітко розмежованим правам

доступу та продуманому функціоналу система забезпечує безпечну та ефективну роботу для всіх користувачів.

У результаті виконання курсової роботи було створено ефективний та зручний інструмент для організації навчального процесу, який може бути основою для подальшого вдосконалення та масштабування.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

- 1) ASP.NET Core [Електронний ресурс] - <https://dotnet.microsoft.com/en-us/apps/aspnet>.
- 2) C# [Електронний ресурс] - <https://learn.microsoft.com/en-us/dotnet/csharp/>
- 3) .NET [Електронний ресурс] - <https://dotnet.microsoft.com/en-us/>
- 4) N-tier architecture [Електронний ресурс] - https://en.wikipedia.org/wiki/Multitier_architecture
- 5) JWT [Електронний ресурс] - <https://jwt.io/>
- 6) MS SQL Server [Електронний ресурс] – <https://learn.microsoft.com/en-us/sql/?view=sql-server-ver16>
- 7) Entity Framework [Електронний ресурс] – <https://learn.microsoft.com/en-us/sql/?view=sql-server-ver16>
- 8) DataGrip [Електронний ресурс] -<https://www.jetbrains.com/datagrip/>
- 9) React [Електронний ресурс] – <https://react.dev/>
- 10) TypeScript [Електронний ресурс] - <https://www.typescriptlang.org/>
- 11) Visual Studio Code [Електронний ресурс] - <https://code.visualstudio.com/>
- 12) Document Object Model [Електронний ресурс] - https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
- 13) Material UI [Електронний ресурс] - <https://mui.com/material-ui/>
- 14) Axios [Електронний ресурс] - <https://axios-http.com/docs/intro>

ДОДАТКИ

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

керівник

_____ Максим ГОЛОВЧЕНКО

“ ____ ” _____ 2024 р.

ВЕБ-ЗАСТОСУВАННЯ ОРГАНІЗАЦІЯ НАВЧАЛЬНИХ КУРСІВ

Текст програми

КПІ.ІІ-23.045440.01.91

“ПОГОДЖЕНО”

Керівник роботи:

Максим ГОЛОВЧЕНКО

Виконавець:

Іван ПОДГОРНИЙ

Посилання на репозиторій з повним текстом програмного коду

<https://github.com/fedyafedko/StudyHub-BE>

<https://github.com/fedyafedko/StudyHub-FE>

Файл AuthService.cs

```
using AutoMapper;
using Microsoft.AspNetCore.Identity;
using StudyHub.Common.DTO.AuthDTO;
using StudyHub.Common.Exceptions;
using StudyHub.DAL.Repositories.Interfaces;
using StudyHub.Entities;
using System.IdentityModel.Tokens.Jwt;
using StudyHub.BLL.Services.Interfaces.Auth;
using StudyHub.Common.Requests;
using System.Web;

namespace StudyHub.BLL.Services.Auth;

public class AuthService : IAuthService
{
    protected readonly UserManager<User> _userManager;
    protected readonly ITokenService _tokenService;
    protected readonly IMapper _mapper;
    protected readonly IRepository<InvitedUser> _invitedUserRepository;
    protected readonly IEncryptService _encryptService;
    private readonly IRepository<RefreshToken> _refreshTokenRepository;

    public AuthService(
        UserManager<User> userManager,
        ITokenService tokenService,
        IEncryptService encryptService,
        IRepository<InvitedUser> invitedUserRepository,
        IRepository<RefreshToken> refreshTokenRepository,
        IMapper mapper)
    {
        _refreshTokenRepository = refreshTokenRepository;
        _invitedUserRepository = invitedUserRepository;
    }
}
```

```

        _tokenService = tokenService;
        _userManager = userManager;
        _tokenService = tokenService;
        _encryptService = encryptService;
        _mapper = mapper;
    }

    public async Task<AuthSuccessDTO> LoginAsync(LoginUserDTO dto)
    {
        var user = await _userManager.FindByEmailAsync(dto.Email)
            ?? throw new NotFoundException($"Unable to find user by specified
email. Email: {dto.Email}");

        var isPasswordValid = await _userManager.CheckPasswordAsync(user,
dto.Password);

        if (!isPasswordValid)
            throw new InvalidCredentialsException($"User input incorrect password.
Password: {dto.Password}");

        return await GetAuthTokensAsync(user);
    }

    public async Task<AuthSuccessDTO> RegisterAsync(RegisterUserDTO dto)
    {
        var invitedUser = _invitedUserRepository.FirstOrDefault(e => e.Email ==
dto.Email)
            ?? throw new NotFoundException($"User with this email wasn't invited:
{dto.Email}");

        if (!_encryptService.Verify(HttpUtility.UrlDecode(dto.Token),
invitedUser.Token))
            throw new InvalidTokenException($"This token isn't correct:
{dto.Token}");

        var user = await _userManager.FindByEmailAsync(dto.Email);

        if (user != null)
            throw new AlreadyExistsException($"User with specified email already

```

```
exists. Email: {dto.Email}");
```

```
user = _mapper.Map<User>(dto);
```

```
var result = await _userManager.CreateAsync(user, dto.Password);
```

```
if (!result.Succeeded)
```

```
    throw new UserManagerException($"User manager operation failed:\n",
result.Errors);
```

```
var role = invitedUser.Role;
```

```
var currentUser = await _userManager.FindByIdAsync(user.Id.ToString());
```

```
var roleResult = await _userManager.AddToRoleAsync(user, role);
```

```
if (!roleResult.Succeeded)
```

```
    throw new UserManagerException($"User manager operation failed:\n",
roleResult.Errors);
```

```
await _invitedUserRepository.DeleteAsync(invitedUser);
```

```
return await GetAuthTokensAsync(user);
```

```
}
```

```
public async Task<AuthSuccessDTO>
RefreshTokenAsync(RefreshTokenRequest request)
```

```
{
```

```
    var validatedToken =
_tokenService.GetPrincipalFromToken(request.AccessToken);
```

```
    var expiryDateUnix = long.Parse(validatedToken.Claims.Single(x => x.Type
== JwtRegisteredClaimNames.Exp).Value);
```

```
    var expiryDateTimeUtc = new DateTime(year: 1970, month: 1, day: 1, hour:
0, minute: 0, second: 0, DateTimeKind.Utc)
```

```
        .AddSeconds(expiryDateUnix);
```

```

        if (expiryDateTimeUtc > DateTime.Now)
            throw new IncorrectParametersException("Access token is not expired yet");

        var jti = validatedToken.Claims.Single(x => x.Type ==
JwtRegisteredClaimNames.Exp).Value;

        var existingToken = _refreshTokenRepository.FirstOrDefault(rf => rf.Token
== request.RefreshToken)
            ?? throw new NotFoundException($"This token doesn't found:
{request.RefreshToken}");

        if (DateTimeOffset.Now > existingToken.ExpiryDate)
            throw new ExpiredException("Refresh token is expired");

        if (existingToken.Invalidated)
            throw new TokenValidatorException($"This token is invalidated:
{request.RefreshToken}");

        await _refreshTokenRepository.DeleteAsync(existingToken);

        var user = await
_userManager.FindByIdAsync(validatedToken.Claims.Single(x => x.Type ==
"id").Value)
            ?? throw new NotFoundException("User with this id does not exist");

        return await GetAuthTokensAsync(user);
    }

    protected async Task<AuthSuccessDTO> GetAuthTokensAsync(User user)
    {
        return new AuthSuccessDTO(await
_tokenService.GenerateJwtTokenAsync(user),
            await _tokenService.GenerateRefreshTokenAsync(user));
    }
}

```

Файл SubjectService.cs
using AutoMapper;

```

using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using StudyHub.BLL.Extensions;
using StudyHub.BLL.Services.Interfaces;
using StudyHub.Common.DTO.Assignment;
using StudyHub.Common.DTO.Subject;
using StudyHub.Common.DTO.User.Student;
using StudyHub.Common.Exceptions;
using StudyHub.Common.Requests;
using StudyHub.Common.Response;
using StudyHub.DAL.Repositories.Interfaces;
using StudyHub.Entities;
using System.Diagnostics;

namespace StudyHub.BLL.Services;

public class SubjectService : ISubjectService
{
    private readonly IRepository<Subject> _subjectRepository;
    private readonly IRepository<StudentSubject> _studentSubjectsRepository;
    private readonly IRepository<StartingTimeRecord>
_startingTimeRecordsRepository;
    private readonly UserManager<User> _userManager;
    private readonly IMapper _mapper;

    public SubjectService(
        IRepository<Subject> subjectRepository,
        UserManager<User> userManager,
        IMapper mapper,
        IRepository<StudentSubject> studentSubjectsRepository,
        IRepository<StartingTimeRecord> startingTimeRecordsRepository)
    {
        _subjectRepository = subjectRepository;
        _userManager = userManager;
        _mapper = mapper;
        _studentSubjectsRepository = studentSubjectsRepository;
        _startingTimeRecordsRepository = startingTimeRecordsRepository;
    }
}

```

```

    public async Task<SubjectDTO> AddSubjectAsync(Guid teacherId,
CreateSubjectDTO dto)
    {
        var teacher = await _userManager.FindByIdAsync(teacherId)
            ?? throw new NotFoundException($"Teacher not found in the database
with this ID: {teacherId}");

        var entity = _mapper.Map<Subject>(dto);

        entity.Teacher = teacher;

        await _subjectRepository.InsertAsync(entity);

        return _mapper.Map<SubjectDTO>(entity);
    }

    public async Task<bool> DeleteSubjectAsync(Guid userId, Guid subjectId)
    {
        var entity = await _subjectRepository
            .FirstOrDefaultAsync(subject => subject.Id == subjectId)
            ?? throw new NotFoundException($"Unable to find entity with such key:
{subjectId}");

        if (entity.TeacherId != userId)
            throw new RestrictedAccessException("You are not the owner and do not
have permission to perform this action.");

        return await _subjectRepository.DeleteAsync(entity);
    }

    public async Task<SubjectDTO> GetSubjectAsync(Guid subjectId)
    {
        var entity = await _subjectRepository
            .Include(s => s.Teacher)
            .FirstOrDefaultAsync(subject => subject.Id == subjectId)
            ?? throw new NotFoundException($"Unable to find entity with such key:
{subjectId}");

```

```

        return _mapper.Map<SubjectDTO>(entity);
    }

    public async Task<List<SubjectDTO>> GetSubjectsForUserAsync(Guid
userId)
    {
        var user = await _userManager.Users
            .Include(u => u.TeacherSubjects)
            .FirstOrDefaultAsync(u => u.Id == userId)
            ?? throw new NotFoundException($"Unable to find entity with such key:
{userId}");

        var roles = await _userManager.GetRolesAsync(user);

        var subjects = roles.First() == "Student"
            ? await _studentSubjectsRepository
                .Include(s => s.Subject)
                .ThenInclude(s => s.Teacher)
                .Where(s => s.StudentId == userId)
                .Select(s => s.Subject)
                .ToListAsync()
            : user.TeacherSubjects;

        return _mapper.Map<List<SubjectDTO>>(subjects);
    }

    public async Task<SubjectDTO> UpdateSubjectAsync(Guid userId, Guid
subjectId, UpdateSubjectDTO dto)
    {
        var entity = await _subjectRepository
            .FirstOrDefaultAsync(subject => subject.Id == subjectId)
            ?? throw new NotFoundException($"Unable to find entity with such key:
{subjectId}");

        if (entity.TeacherId != userId)
            throw new RestrictedAccessException("You are not the owner and do not
have permission to perform this action.");
    }

```

```

_mapper.Map(dto, entity);

await _subjectRepository.UpdateAsync(entity);

return _mapper.Map<SubjectDTO>(entity);
}

public async Task<StudentResultResponse> AddStudentsToSubjectAsync(
    Guid subjectId,
    Guid teacherId,
    StudentsToSubjectRequest request)
{
    var subject = await _subjectRepository.FirstOrDefaultAsync(s => s.Id ==
subjectId)
        ?? throw new NotFoundException($"Subject not found with this ID:
{subjectId}");

    if (subject.TeacherId != teacherId)
        throw new RestrictedAccessException("You are not the owner and do not
have permission to perform this action.");

    var response = new StudentResultResponse();

    foreach (var email in request.Emails)
    {
        var user = await _userManager.FindByEmailAsync(email);

        if (user == null)
        {
            response.Failed.Add(email);
            continue;
        }

        var entity = new StudentSubject
        {
            StudentId = user.Id,
            SubjectId = subject.Id

```



```

        };

        await _studentSubjectsRepository.InsertAsync(entity);
        response.Success.Add(email);
    }

    return response;
}

public async Task<bool> DeleteStudentsFromSubjectAsync(
    Guid subjectId,
    Guid teacherId,
    string studentEmail)
{
    var subject = await _subjectRepository
        .FirstOrDefaultAsync(s => s.Id == subjectId)
        ?? throw new NotFoundException($"Subject not found with this ID:
{subjectId}");

    if (subject.TeacherId != teacherId)
        throw new RestrictedAccessException("You are not the owner and do not
have permission to perform this action.");

    var user = await _userManager.FindByEmailAsync(studentEmail)
        ?? throw new NotFoundException($"Student not found with current Email:
{studentEmail}");

    var entity = await _studentSubjectsRepository
        .FirstOrDefaultAsync(s => s.StudentId == user.Id && s.SubjectId ==
subject.Id)
        ?? throw new NotFoundException($"Student not exist in current subject:
{studentEmail}");

    return await _studentSubjectsRepository.DeleteAsync(entity);
}

public async Task<List<StudentDTO>> GetStudentsForSubjectAsync(Guid
subjectId)

```

```

{
    var students = await _studentSubjectsRepository
        .Include(s => s.Student)
        .Where(s => s.SubjectId == subjectId)
        .Select(s => s.Student)
        .ToListAsync()
        ?? throw new NotFoundException($"Subject not found with this ID:
{subjectId}");

    return _mapper.Map<List<StudentDTO>>(students);
}

public async Task<bool> AddMarkForStudent(Guid teacherId,
MarkForSubjectRequest request)
{
    var subject = await _subjectRepository
        .FirstOrDefaultAsync(s => s.Id == request.SubjectId)
        ?? throw new NotFoundException($"Subject not found with this ID:
{request.SubjectId}");

    if (subject.TeacherId != teacherId)
        throw new RestrictedAccessException("You are not the owner and do not
have permission to perform this action.");

    var studentSubject = await _studentSubjectsRepository
        .FirstOrDefaultAsync(s => s.StudentId == request.StudentId &&
s.SubjectId == request.SubjectId)
        ?? throw new NotFoundException($"The student does not belong to this
subject");

    studentSubject.Mark = request.Mark;

    var result = await _studentSubjectsRepository.UpdateAsync(studentSubject);

    return result;
}

public async Task<List<StudentMarksResponse>>
GetMarksForUserAsync(Guid studentId)

```

```

{
    var student = await _userManager.FindByIdAsync(studentId)
        ?? throw new NotFoundException($"Student not found with this ID:
{studentId}");

    var studentSubjects = await _studentSubjectsRepository
        .Include(s => s.Subject)
        .ThenInclude(s => s.Assignments)
        .Where(s => s.StudentId == studentId)
        .ToListAsync();

    var result = new List<StudentMarksResponse>();

    foreach (var studentSubject in studentSubjects)
    {
        var response = new StudentMarksResponse
        {
            Subject = _mapper.Map<SubjectDTO>(studentSubject.Subject),
            SubjectMark = Math.Round(studentSubject.Mark),
            AssignmentMarks = studentSubject.Subject.Assignments
                .Select(a =>
                {
                    var record = _startingTimeRecordsRepository
                        .FirstOrDefault(x => x.StudentId == studentId &&
x.AssignmentId == a.Id);

                    return new AssignmentMarkDTO
                    {
                        Assignment = _mapper.Map<AssignmentDTO>(a),
                        Mark = record != null ? Math.Round(record.Mark) : 0
                    };
                })
                .ToList()
        };
        result.Add(response);
    }

    return result;
}

```

```

    }

    public async Task<StudentsSubjectMarksResponse>
    GetStudentMarksForTeacherAsync(Guid subjectId)
    {
        var studentSubject = await _studentSubjectsRepository
            .Include(s => s.Student)
            .Include(s => s.Subject)
            .ThenInclude(s => s.Assignments)
            .Where(s => s.SubjectId == subjectId)
            .ToListAsync();

        var studentAssingmentMarks = new List<StudentAssingmentMarks>();

        foreach (var item in studentSubject)
        {
            var studentMarks = new StudentAssingmentMarks
            {
                AssignmentMarks = item.Subject.Assignments
                    .Select(a =>
                    {
                        var record = _startingTimeRecordsRepository
                            .FirstOrDefault(x => x.StudentId == item.StudentId &&
                                x.AssignmentId == a.Id);

                        return new AssignmentMarkDTO
                        {
                            Assignment = _mapper.Map<AssignmentDTO>(a),
                            Mark = record != null ? Math.Round(record.Mark) : 0
                        };
                    })
                    .ToList(),
                StudentName = item.Student.FullName,
                SubjectMark = item.Mark
            };

            studentAssingmentMarks.Add(studentMarks);
        }
    }

```

```

    var result = new StudentsSubjectMarksResponse
    {
        Title = studentSubject.First().Subject.Title,
        StudentAssingmentMarks = studentAssingmentMarks
    };

    return result;
}
}

```

Файл SubjectPage.tsx

```

import { Box, Button, Typography } from "@mui/material";
import { useNavigate, useParams } from "react-router-dom";
import ButtonsMenu from "../components/ButtonsMenu/ButtonsMenu";
import { useEffect, useState } from "react";
import User from "../api/User";
import UserResponse from "../api/models/response/UserResponse";
import styles from './SubjectPage.module.css';
import profile from '../img/Profile.png';
import Subject from "../api/Subject";
import AssignmentResponse from
"../api/models/response/AssignmentResponse";
import SubjectResponse from "../api/models/response/SubjectResponse";
import PageHeader from "../components/PageHeader/PageHeader";
import AssignmentsCard from
"../components/AssignmentCard/AssignmentCard";
import AddStudentsComponent from
"../components/AddStudentsToSubject/AddStudentsComponent";

const SubjectPage = () => {
    const { id } = useParams();
    const [user, setUser] = useState<UserResponse | undefined>(undefined);
    const [assignments, setAssignments] = useState<AssignmentResponse[] |
undefined>([]);
    const [subject, setSubject] = useState<SubjectResponse | undefined>();
    const navigate = useNavigate();

    useEffect(() => {

```

```

const fetchUser = async () => {
  try {
    const response = await User.me();
    setUser(response);
  } catch (error) {
    console.error(error);
    setUser(undefined);
  }
};

fetchUser();
console.log(user);
}, []);

useEffect(() => {
  const fetchAssignments = async () => {
    try {
      const response = await Subject.getSubjectWithAssignment(id);
      setAssignments(response);
    } catch (error) {
      console.error(error);
      setAssignments([]);
    }
  };
  fetchAssignments();
}, []);

useEffect(() => {
  const fetchSubject = async () => {
    try {
      const response = await Subject.getSubject(id);
      setSubject(response);
    } catch (error) {
      console.error(error);
    }
  };

  fetchSubject();

```

}, [])

```

return (
  <div>
    <PageHeader user={user} />
    <div className={styles.pageBox}>
      <ButtonsMenu activeView="subjects" user={user} />
      <div className={styles.contentBox}>
        <div style={{ display: 'flex', flexDirection: 'column', marginLeft: '2rem' }}>
          <div style={{ display: 'flex', flexDirection: 'row', justifyContent: "space-between", alignItems: "center" }}>
            <Typography variant="h4" sx={{ fontWeight: 'bold' }}>{subject?.title}</Typography>
            <Button
              variant="outlined"
              size="medium"
              sx={{ marginTop: "1rem" }}
              onClick={() => navigate(`/subject/${subject?.id}/performance`)}
            >
              Performance
            </Button>
          </div>
          <AssignmentsCard assignments={assignments} user={user}
subjectId={subject?.id} onNavigate={navigate} />
        </div>
        <div style={{ display: 'flex', flexDirection: "column" }}>
          <div className={styles.teacherBox}>
            <Box component='img' sx={{ width: '90px', height: '90px' }}
src={profile} />
            <div style={{ marginLeft: '20px' }}>
              <Typography variant="h5" sx={{ fontWeight: 'bold' }}>{subject?.teacher.fullName}</Typography>
              <Typography variant="h6" sx={{ fontSize: "0.95rem", color: "gray" }}>{subject?.teacher.email}</Typography>
            </div>
          </div>
        </div>
        {user?.role.toLowerCase() === 'teacher' && <AddStudentsComponent />}
        {user?.role.toLowerCase() === 'student'}
      </div>
    </div>
  </div>
)

```

```

        </div>
      </div>
    </div>
  </div>
);
};

```

```
export default SubjectPage;
```

Файл AssignmentTaskPage.tsx

```

import React, { useState, useEffect } from 'react';
import { Box, Button, Typography, CircularProgress, Dialog, DialogTitle,
DialogContent, DialogContentText, DialogActions } from '@mui/material';
import UserResponse from '../api/models/response/UserResponse';
import User from '../api/User';
import ButtonsMenu from '../components/ButtonsMenu/ButtonsMenu';
import PageHeader from '../components/PageHeader/PageHeader';
import CountdownTimer from
'../components/TestComponents/CountdownTimer';
import Assignment from '../api/Assignment';
import QuestionForTest from '../api/models/response/QuestionForTest';
import { mapTestResponseToQuestion } from
'../api/mapper/mapTestResponseToQuestion';
import { useNavigate, useParams } from 'react-router-dom';
import Cookies from "js-cookie";
import QuestionRenderer from
'../components/TestComponents/QuestionRender';
import { AnswerVariant, SendTestAnswers } from
'../api/models/request/Assignment/SendTestAnswers';

interface Answer {
  guid: string | string[];
}

```

```

const AssignmentTaskPage: React.FC = () => {
  const { id: assignmentPathId } = useParams();
  const assignmentId = assignmentPathId?.split('/').pop();
  const [user, setUser] = useState<UserResponse | undefined>(undefined);

```



```

const [currentQuestionIndex, setCurrentQuestionIndex] = useState<number>(0);
const [answers, setAnswers] = useState<Answer[]>([]);
const [selectedOptions, setSelectedOptions] = useState<string[]>([]);
const [questions, setQuestions] = useState<QuestionForTest[]>([]);
const [loading, setLoading] = useState(true);
const [isDialogOpen, setIsDialogOpen] = useState(false);
const navigate = useNavigate();

const fetchUser = async () => {
  try {
    const userResponse = await User.me();
    setUser(userResponse);
  } catch (error) {
    console.error(error);
    setUser(undefined);
  }
};

const fetchTest = async () => {
  const storedData = JSON.parse(Cookies.get("TestData") || "[]") as
QuestionForTest[];

  if (!storedData.length) {
    try {
      const testResponse = await
Assignment.getAssignmentTest(assignmentId);
      if (testResponse) {
        const mappedQuestions =
mapTestResponseToQuestion(testResponse);
        Cookies.set("TestData", JSON.stringify(mappedQuestions), { expires:
0.5 });
        setQuestions(mappedQuestions);
      }
    } catch (error) {
      console.error("Error fetching test data:", error);
    }
  } else {
    setQuestions(storedData);
  }
}

```

```

    }
  };

const fetchData = async () => {
  setLoading(true);

  await Promise.all([fetchTest(), fetchUser()]);
  console.log(questions);
  setLoading(false);
};

useEffect(() => {
  fetchData();
}, [assignmentId]);

const handleBoxClick = (option: string, isMulti: boolean, quid: string) => {
  if (isMulti) {
    setSelectedOptions((prev) => {
      if (prev.includes(option)) {
        return prev.filter((o) => o !== option);
      } else {
        return [...prev, option];
      }
    });
  } else {
    handleChange(questions[currentQuestionIndex].id, option);
    setSelectedOptions([option]);
  }
};

useEffect(() => {
  if (selectedOptions.length > 0) {
    const options = questions[currentQuestionIndex].options;
    const matchingOptions = options!.filter((option) =>
selectedOptions.includes(option.label));
    handleChange(questions[currentQuestionIndex].id,
matchingOptions.map((option) => option.id));
  }

```

```
}, [selectedOptions]);
```

```
const handleNextQuestion = (index: number) => () => {
  setSelectedOptions([]);
  setCurrentQuestionIndex(index);
};
```

```
const handleFinishTest = async () => {
  const answerVariants: AnswerVariant[] = questions.map((question) => ({
    taskVariantId: question.guid,
    taskOptionIds: answers[question.id].guid,
  }));
```

```
const payload: SendTestAnswers = {
  assignmentId: assignmentId!,
  answerVariants: answerVariants
}
```

```
try {
  const result = await Assignment.uploadStudentAnswers(payload);
  Cookies.remove("TestData");
  navigate(`/dashboard`);
} catch (error) {
  console.error(error);
}
```

```
console.log('Test Finished:', payload);
};
```

```
const handleChange = (questionId: number, guid: string | string[]) => {
  const updatedAnswer: Answer = {
    guid: guid,
  };
};
```

```
setAnswers((prevAnswers) => ({
  ...prevAnswers,
  [questionId]: updatedAnswer,
}));
```

```

};

if (loading) {
  return (
    <Box
      sx={{
        display: 'flex',
        justifyContent: 'center',
        alignItems: 'center',
        height: '100vh',
      }}
    >
    <CircularProgress />
  </Box>
);
}

const handleDialogCancel = () => {
  setIsDialogOpen(false);
}

return (
  <div>
    <Dialog open={isDialogOpen} onClose={handleDialogCancel}>
      <DialogTitle>Unsaved Changes</DialogTitle>
      <DialogContent>
        <DialogContentText>
          You have unsaved changes. Are you sure you want to leave this
page? Your progress will be lost.
        </DialogContentText>
      </DialogContent>
      <DialogActions>
        <Button onClick={handleDialogCancel} color="primary">
          Stay on Page
        </Button>
        <Button onClick={handleFinishTest} color="secondary">
          Leave Page
        </Button>
      </DialogActions>
    </Dialog>
  </div>
);

```

```

    </DialogActions>
  </Dialog>
  <PageHeader user={user} />
  <div style={{ display: 'flex', flexDirection: 'row' }}>
    <ButtonsMenu activeView="subjects" user={user} />
    <div style={{ display: 'flex', flexDirection: 'column', alignItems: 'center',
justifyContent: 'center', marginLeft: '10%', gap: '2rem' }}>
      <Typography variant="h4" sx={{ fontWeight: 'normal'
}}>{questions[currentQuestionIndex].question}</Typography>
      <div>
        <QuestionRenderer
          question={questions[currentQuestionIndex]}
          selectedOptions={selectedOptions}
          handleBoxClick={handleBoxClick}
          handleChange={handleChange}
          answers={answers}
        />
      </div>
      <div style={{ marginTop: '2rem', display: 'flex', justifyContent:
'space-between', width: '100%' }}>
        <Button variant="contained" color="primary"
onClick={handleNextQuestion(currentQuestionIndex - 1)}
disabled={currentQuestionIndex === 0}>
          Previous Question
        </Button>
        {currentQuestionIndex < questions.length - 1 ? (
          <Button variant="contained" color="primary"
onClick={handleNextQuestion(currentQuestionIndex + 1)}>
            Next Question
          </Button>
        ) : (
          <Button variant="contained" color="secondary"
onClick={handleFinishTest}>
            Finish Test
          </Button>
        )}
      </div>
    </div>
  </div>
  <div style={{ display: "flex", flexDirection: "column", alignItems:

```

```

"center", width: "30%", marginTop: "4%" }}>
  <CountdownTimer />
  <div style={{ marginTop: '1rem', display: 'flex', gap: '0.5rem' }}>
    {questions.map((_, index) => (
      <div
        key={index}
        onClick={handleNextQuestion(index)}
        style={{
          padding: '10px 20px',
          border: '1px solid',
          borderColor: "#ddd",
          borderRadius: '5px',
          cursor: 'pointer',
          backgroundColor: currentQuestionIndex === index ?
            '#ffd3f8' : 'white',
          textAlign: 'center',
          position: 'relative',
        }}
      >
        {index + 1}
        {answers[index + 1] &&
          <div
            style={{
              position: 'absolute',
              bottom: '0',
              left: '50%',
              transform: 'translateX(-50%) translateY(50%)',
              width: 0,
              height: 0,
              borderLeft: '10px solid transparent',
              borderRight: '10px solid transparent',
              borderTop: `10px solid rgb(18, 163, 253)`,
            }}
          />
        }
      </div>
    ))}
  </div>

```

```
        </div>
      </div>
    </div>
  );
};

export default AssignmentTaskPage;
```